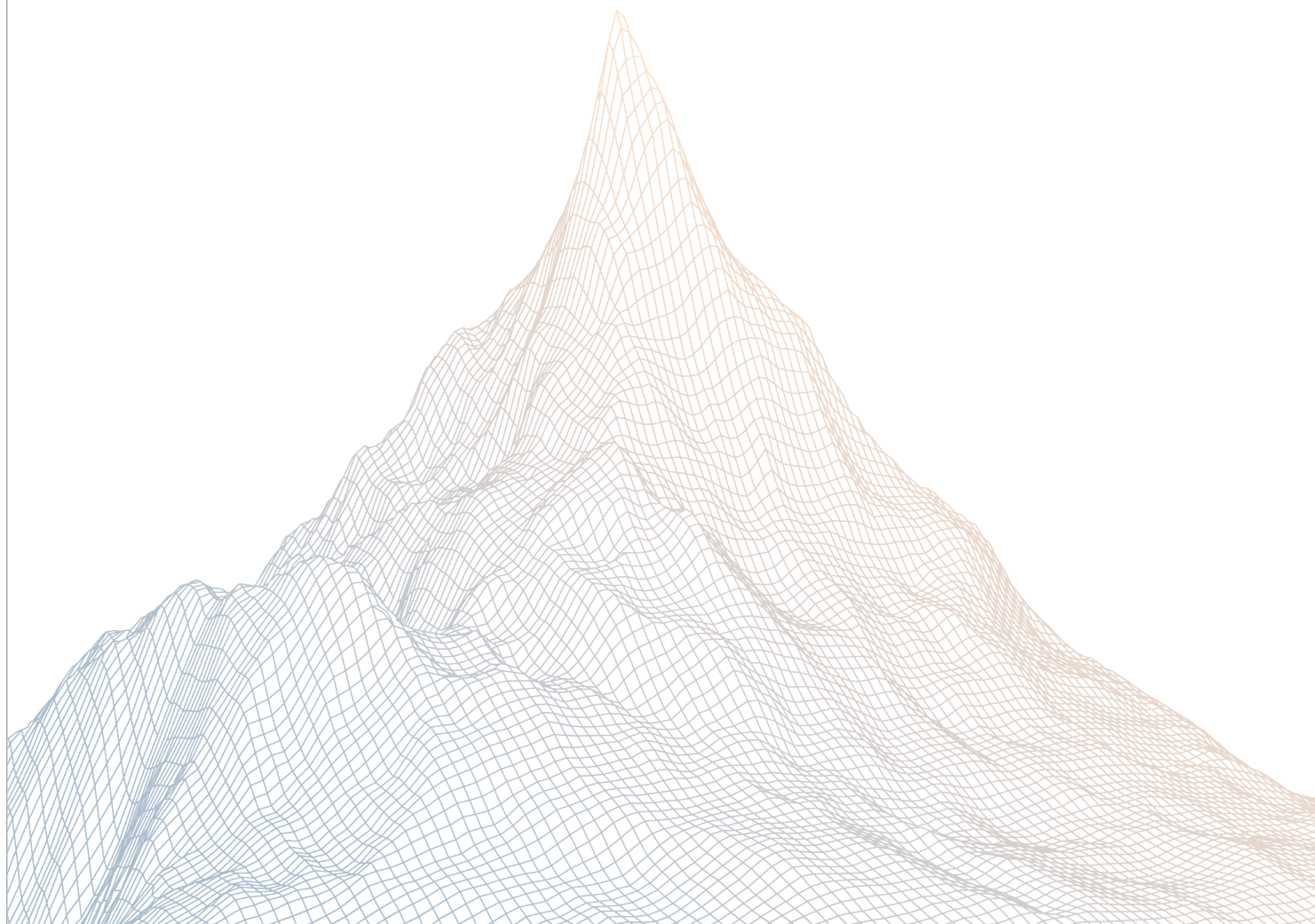


Techdollar

Application and Infrastructure Security Assessment

VERSION 1.1



AUDIT DATES:

July 6th to July 24th, 2026

AUDITED BY:

Jakub Heba
Matte

Contents

1	Introduction	3
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	4
2.1	About Techdollar	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
2.5	Security Note	6
3	Findings Summary	7
4	Findings	12
4.1	High Risk	12
4.2	Medium Risk	18
4.3	Low Risk	60
4.4	Informational	117

1

Introduction

1.1

About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry’s top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2

Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3

Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1

About Techdollar

Providing private equity shareholders with liquidity for quality of life improvements, while preserving their upside

2.2

Scope

The engagement involved a review of the following targets:

Target	back
Repository	https://github.com/techdollar/back
Commit Hash	d20b2cc40b5c0cf2e2340ff1e480b528af683be8
Files	src/**/*
Target	interface
Repository	https://github.com/techdollar/interface
Commit Hash	c53c47f40e297278c1d11e336e07513b466d7bd4
Files	**/*

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/techdollar/back
-------------------	---

Commit Hash	ff38718d8e095069eee9609cb5eec00b05bb5af3
--------------------	--

Files	src/**/*
--------------	----------

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/techdollar/interface
-------------------	---

Commit Hash	b33fb7076f6792112b04230cdae9bef9f1a15722
--------------------	--

Files	**/*
--------------	------

2.3 Audit Timeline

July 6, 2026	Audit start
July 24, 2026	Audit end
August 21, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	4
Medium Risk	25
Low Risk	36
Informational	6
Total Issues	71

2.5 Security Note

Considering the number of issues identified, it is statistically likely that there are more complex bugs still present that could not be identified given the time-boxed nature of this engagement. It is recommended that a follow-up audit and development of a more complex stateful test suite be undertaken prior to continuing to deploy significant monetary capital to production.

3

Findings Summary

ID	Description	Status
H-1	Sumsb webhook status monotonicity failure in handleWebhook	Resolved
H-2	UR managed custody server actions trust a caller-supplied urlId	Resolved
H-3	Final bullet repayment is submitted through the interest-only repayment function	Resolved
H-4	Authorization latency amplification through KycGuard	Resolved
M-1	transactionId idempotency invariant missing for banking payments	Resolved
M-2	updateApplication atomicity failure across application and loan terms	Resolved
M-3	computeFullRepayAmount payoff invariant omits final interest	Resolved
M-4	updateApplication auditor mutation invariant break	Resolved
M-5	LoanOnchainRepaymentSyncService progress invariant failure past 200 loans	Resolved
M-6	KYC state regression through stale User persistence	Resolved
M-7	EquityApplicationsService upload path erases approval state	Resolved
M-8	Borrower loan endpoint leaks admin internal fields	Resolved
M-9	Onchain authorization registered with the uncapped amount	Resolved
M-10	Turnkey session JWT not bound to the expected organization	Resolved
M-11	UR_TESTNET_URL and SUBGRAPH_URL accept plaintext authority endpoints	Resolved
M-12	Sumsb action tokens use the provider's internal ID as externalActionId	Resolved

ID	Description	Status
M-13	Explicit 0% interest overrides fall back to the company or default rate	Resolved
M-14	Public webhook callers share a rate-limit bucket behind the ALB	Resolved
M-15	addApplicationComment bounded-write validation gap	Resolved
M-16	getPagination amplification through unchecked page	Resolved
M-17	UpcomingLoansService projection invariant exposes raw Company	Resolved
M-18	walletAddress canonicalization invariant missing from linkWallet	Resolved
M-19	On-chain repayment sync stops ingesting after the first 100 subgraph events	Resolved
M-20	getTVL reserve-direction invariant failure in header accounting	Resolved
M-21	Uncursored take semantics block newer loan activation	Resolved
M-22	allocateInterestPayment state writes precede Payment uniqueness	Resolved
M-23	MAX_FILE_SIZE post-parse validation gap	Resolved
M-24	loan-onchain-repayment-sync status-gated recovery flaw	Resolved
M-25	Cron exclusivity invariant failure in runExclusive	Resolved
L-1	Concurrent create-account calls orphan UR accounts	Resolved
L-2	confirmApprovedApplication uniqueness invariant missing database enforcement	Resolved
L-3	buildDbSslConfig transport-security invariant failure	Resolved

ID	Description	Status
L-4	EVM settlement state transition without amount equality	Resolved
L-5	ILike literal-search invariant failure	Resolved
L-6	LoanVerifier registration consistency loss in confirmApprovedApplication	Resolved
L-7	Open redirect on /kyc via unvalidated callbackUrl	Resolved
L-8	Wallet address linkable without proof of ownership	Resolved
L-9	Month-end schedule generation creates overlapping payment periods	Resolved
L-10	Concurrent distinct banking receipts lose loan accounting updates	Resolved
L-11	Historical settlement events can be reused to confirm later payments	Resolved
L-12	getLoanAmountByCompany predicate mismatch on company visibility	Resolved
L-13	getPoints pagination work invariant failure	Resolved
L-14	safeTxHash canonicalization invariant failure	Resolved
L-15	response.Body pipe without source error containment	Resolved
L-16	handleAccountResult live account consistency regression	Resolved
L-17	process.env.DB_SSL raw-value validation mismatch	Resolved
L-18	getDashboard cardinality invariant failure for borrower applications	Resolved
L-19	evmAddressSchema recipient validation invariant omits zero-address rejection	Resolved

ID	Description	Status
L-20	Turnkey and Sumsb identity data written to production info logs	Resolved
L-21	Borrower can cancel an equity application after the loan has been created	Resolved
L-22	Interest overpayment beyond the full remaining schedule is recorded but not allocated	Resolved
L-23	S3 object key derived from deterministic inputs allows a same millisecond collision	Resolved
L-24	Loan initiation sync activates a loan without checking the event type against loan.isOnchain	Resolved
L-25	updateProfile authority invariant broken for custody identifiers	Resolved
L-26	Error-state consistency failure in usePaginatedApiQuery	Resolved
L-27	Transaction-display consistency failure in toBigNumberish	Resolved
L-28	handleSessionExpired query-isolation invariant failure	Resolved
L-29	Quote-state consistency failure in Swap	Resolved
L-30	Payoff principal-argument inconsistency in PayPrincipal	Resolved
L-31	Authorization inconsistency between Unstake and requestWithdrawal()	Resolved
L-32	urActionStatus consistency loss in UrOnboardingService	Resolved
L-33	verifySignature accepts replayable raw-body signatures	Resolved
L-34	LoansReadService top-level pagination bypassed by nested payments	Resolved
L-35	Loan confirmation state transition without KycGuard	Resolved

ID	Description	Status
L-36	Loan activation is committed before repayment schedule creation	Resolved
I-1	Turnkey linking consistency loss through updateProfile	Resolved
I-2	linkedin scheme validation invariant failure	Resolved
I-3	maxLtvPercent policy invariant violation	Resolved
I-4	SubgraphClientService timeout invariant loss	Resolved
I-5	APP_GUARD ordering weakens authentication throttling	Resolved
I-6	Route-schema validation absence in querySwapRoute	Resolved

4

Findings

4.1 High Risk

A total of 4 high risk findings were identified.

[H-1] Sumsub webhook status monotonicity failure in handleWebhook

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [sumsub.service.ts](#)
- [ur-onboarding.service.ts](#)

Description

Provider-signed compliance events should be accepted only when they are fresh and bound to the current applicant or action state. `handleWebhook()` updates `kycStatus` from `externalUserId`, while `handleActionWebhook()` updates `urActionStatus` from the same user lookup; I traced both paths and found no comparison against `createdAtMs`, `applicantActionId`, `externalActionId`, or stored `user.urActionId` (despite storing it earlier).

```
if (payload.type === 'applicantActionReviewed') {
  const answer = payload.reviewResult?.reviewAnswer;
  const rejectType = payload.reviewResult?.reviewRejectType;

  let newStatus: UrActionStatus;
  if (answer === 'GREEN') {
    newStatus = 'verified';
  } else if (answer === 'RED' && rejectType === 'FINAL') {
```

If a user first receives a GREEN event and later receives a FINAL RED event, replaying the older delivery leaves 2 > 0 state variables exposed to stale rollback. KYC or UR verification can be restored to `verified` without forging Sumsub's signature, which corrupts downstream onboarding gates.

Recommendation

Persist provider event identifiers or timestamps and reject stale, duplicate, or non-monotonic transitions. Bind action webhooks to the active user.`urActionId` before mutating `urActionStatus`.

Techdollar: Resolved with [@a4db3c1 ...](#)

Zenith: Verified.

[H-2] UR managed custody server actions trust a caller-supplied urId

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [ur.action.ts](#)
- [fetch.ts](#)

Description:

Every UR server action accepts `urId` as a plain first argument from the caller and forwards it unchanged to `fetchUR`. The `fetchUR` builds its authentication exclusively from the partner private key (`UR_PARTNER_PRIVATE_KEY`, EIP-191 `signMessage` at `fetch.ts:80-85`) and the client-controlled `X-Ur-Id: <urId>` header. It never calls `cookies()`, never reads `AUTH_COOKIE_NAME`, and performs no check that the authenticated caller owns `urId`, so a search for cookie, session, auth or Bearer across `actions/external/ur/` returns nothing.

This is the deliberate opposite of `utils/fetch-secure.ts:47-60`, which reads the `techdollar-auth` cookie and forwards the user's own Bearer token so the backend can scope the request per user. The UR path omits that binding entirely. The middleware provides no compensating control as its matcher is only `/dashboard/*` and `/auth/*`, and even when it runs, `isTokenValid` checks only token expiry, never comparing the JWT `user_id` to `urId`.

At the interface layer nothing binds the request to the caller as the only per-account discriminator is the `urId` value itself, which appears inside `URTransaction` in `types/ur.ts:135` and `URAccount` in `types/ur.ts:292` objects. A live external attack additionally requires discovering a callable UR action ID, which is not currently emitted into client chunks, and the UR backend not independently binding the partner key to a specific `urId`.

The `fetch.ts` is itself a 'use server' module that exports `fetchUR`, so `fetchUR` is registered as its own server action so anyone who learns its action ID can proxy arbitrary UR API paths (any GET or POST request supported by `FetchUROptions`, including with `skipAuth`) under the partner signature, not just the thirteen wrapped endpoints. The `getURPayoutFees` already calls `fetchUR` with `skipAuth: true`, confirming the helper is designed to be driven with attacker-shaped options.

The one limitation is reachability as because no client component imports `URQuery`, the UR action IDs are not currently emitted into client chunks, so an external attacker cannot yet discover a callable ID easily.

Recommendations:

Never accept `urId` or `externalUserId` from the client. Inside each UR action, authenticate the caller server-side by reading the `techdollar-auth` cookie as `fetch-secure.ts` does, resolve the user via `getMe()` or the backend, and use that authenticated `User.urId`. Reject or ignore any client-supplied `urId`.

Techdollar: Resolved with [@afbee9b...](#)

Zenith: Verified.

[H-3] Final bullet repayment is submitted through the interest-only repayment function

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [helpers.ts](#)

Description:

The `generatePaymentSchedule` combines interest, principal, and the exit fee in the final payment's `totalAmount`. The backend stores this value as the scheduled payment amount and later exposes it as `nextPaymentAmount`.

When the final payment becomes the next open obligation, the frontend displays the combined amount as "Interest due" and passes it to `repayOnchainLoanInterest`. The current contract accepts the complete amount and records it as interest without reducing the remaining principal.

If the borrower accepts the prefilled amount and confirms the transaction, they can transfer approximately the entire principal and exit fee as interest while the original principal remains outstanding. This requires sufficient balance, allowance, and borrower confirmation, but the resulting financial loss can be achieved by the attacker.

Recommendations:

Use the scheduled `interestAmount` rather than `nextPaymentAmount` in the Pay Interest flow. Route the final payment through the correct payoff function and add a contract-level limit that prevents interest payments from exceeding unpaid contractual interest.

Techdollar: Resolved with [@266fc8c ...](#)

Zenith: Verified.

[H-4] Authorization latency amplification through KycGuard

SEVERITY: High

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High.

Target

- [kyc.guard.ts](#)
- [sumsub.service.ts](#)

Description

Authorization for unverified users must not synchronously perform uncached third-party polling on every denied request. I traced KycGuard into `refreshStatusFromSumsub`; every non-verified user waits on a live Sumsub GET, and the axios client is created with only `baseUrl`, so no local timeout constrains slow provider behavior (the catch runs after the cost is paid). There is no cooldown. If 512 unverified requests arrive while each Sumsub call stalls, then $512 > 0$ outbound waits accumulate before the guard rejects them. That lets ordinary authenticated users spend backend concurrency and provider quota from routes that should fail cheaply.

```
const updatedStatus = await
  this.sumsubService.refreshStatusFromSumsub(user);
if (updatedStatus === 'verified') {
  const refreshedUser = await this.userRepository.findOne({ where: { id:
    user.id } });
  if (refreshedUser) {
    Object.assign(user, refreshedUser);
    return true;
  }
}
```

Recommendation

Move Sumsub refreshes behind a short timeout, cache or cooldown failed/non-terminal lookups, and let the guard deny from local state when no fresh refresh is permitted. Web-hooks should remain the primary state transition path.

Techdollar: Resolved with [@ee043a6...](#)

Zenith: Verified.

4.2 Medium Risk

A total of 25 medium risk findings were identified.

[M-1] `transactionId` idempotency invariant missing for banking payments

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan.service.ts](#)
- [payment.entity.ts](#)
- [confirm-loan-payment.dto.ts](#)

Description

A banking reference should be consumed at most once across admin payment confirmations. The code does enforce uniqueness for `safeTxHash`, but `transactionId` is stored as a plain nullable column, not a unique banking reference:

- [payment.entity.ts#L61-L70](#)
- [payment.entity.ts#L108-L113](#)

The interest-confirmation path is `adminConfirmInterestPayment`. It checks for an existing banking payment with the same `transactionId`, then allocates interest and saves a new completed payment inside the same transaction:

- [loan.service.ts#L404-L428](#)
- [loan.service.ts#L430-L459](#)

The principal-confirmation path performs the same read-before-write check before saving the completed repayment:

- [loan.service.ts#L519-L543](#)
- [loan.service.ts#L572-L604](#)

This is admin-only and requires two distinct `safeTxHash` values, so it should not be framed as a public exploit. The remaining issue is atomicity: two concurrent admin confirmations using

the same banking `transactionId` but different Safe hashes can both observe no existing row before either insert commits. That can leave two completed payment records tied to one off-chain banking reference, advancing loan repayment state more than the actual cash movement supports.

Recommendation

Enforce banking-reference idempotency at the database layer, for example with a unique constraint or partial unique index over banking payments by `transaction_id`. Keep the application-level check for clearer errors, but do not rely on it as the only guard. Alternatively, take a transaction-scoped lock on the banking reference before allocation and save.

Techdollar: Resolved with [@945ba6a ...](#)

Zenith: Verified.

[M-2] updateApplication atomicity failure across application and loan terms

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [admin-applications.service.ts](#)

Description

Application and loan financing terms should change atomically, but `updateApplication` saves the `EquityApplication` before it checks whether the existing loan already has payments. I traced `AdminController.updateApplication -> AdminApplicationsService.updateApplication`; `termMonths`, `numberOfPayments`, and `isOnchain` enter updates, the application row is persisted, and only then does the loan guard return an error (too late for rollback).

```
Object.assign(application, updates);
await this.equityApplicationRepository.save(application);
this.logger.log(
  `Application ${application.id} updated by admin ${adminId}` +
  `${statusChanged ? ` (status → ${status})` : ''}` +
  `${businessFieldChanged ? ` (business fields changed)` : ''}`,
);
```

With one confirmed payment and a request changing `termMonths` from 12 to 24, `24 > 12` persists on the application even when the loan-side check rejects the change. Borrower and admin read models can then display application terms that no longer match the loan ledger.

Recommendation

Move all loan-dependent validation before the application save, or wrap both entity updates in a transaction that throws on rejected loan changes so neither row commits alone.

Techdollar: Resolved with [@ce75922 ...](#)

Zenith: Verified.

[M-3] computeFullRepayAmount payoff invariant omits final interest

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [helpers.ts](#)
- [loan.service.ts](#)
- [loan.mapper.ts](#)

Description

A full principal payoff must include every open interest component that will be extinguished by closing the loan. `computeFullRepayAmount()` calculates `unpaidInterest` and `currentPeriodInterest`, but the returned `fullRepayAmount` excludes both; I traced the quote into `confirmPrincipalRepayment()` and then `applyPrincipalPaymentAllocationTx()`, where the loan can be set repaid and remaining rows are cancelled (the schedule already carries final interest). For a final row with 1024 principal, 64 interest, and 32 exit fee, the quoted payment becomes $1024 + 32 = 1056$ instead of 1120, so 64 units are waived when the principal reaches zero. The backend then records repayment and clears normal collection state for interest that was never paid.

```
const { interestShortfall, exitFee } = computeContractRepayCharges(loan,
    principalRemaining, asOfDate);

return {
    principalRemaining: round6(principalRemaining),
    unpaidInterest: round6(unpaidInterest),
    currentPeriodInterest: round6(currentPeriodInterest),
    interestShortfall: round6(interestShortfall),
    exitFee: round6(exitFee),
```

Recommendation

Include unpaid and current-period interest in the payoff amount or enforce that the final schedule row interest is settled before principal closure. Keep the borrower quote, admin confirmation, and allocation helper on one accounting invariant.

Techdollar: Resolved with [@2f0c9ff...](#)

Zenith: Verified.

[M-4] updateApplication auditor mutation invariant break

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [admin.controller.ts](#)
- [roles.guard.ts](#)
- [admin-applications.service.ts](#)

Description

Application and loan-term mutations must require administrator authority, but PUT /admin/applications/:id inherits the controller-level auditor role because it has no method-level @Roles('admin'). I traced RolesGuard.getAllAndOverride() into AdminController.updateApplication(); the guard accepts auditor, then the service consumes status, approvedLoanAmount, termMonths, numberOfPayments, rates, fees, and on-chain selection as trusted mutation input (no second authorization gate exists).

```
@Put('applications/:id')
@HttpCode(HttpStatus.OK)
@ApiOperation({
  summary: 'Update application',
  description:
    'Admin update for an equity application. Supports both workflow transitions (status, comments, review metadata) and direct edits to loan-shaping business fields (`approvedLoanAmount`, `termMonths`, `numberOfPayments`, `interestRatePercent`, `exitFeePercent`, `shareClass`, `numberOfShares`, `isOnchain`). `updatedByAdmin` flips to `true` only when one of the business fields actually changes.',
})
```

The role boundary collapses. One auditor account with 0 admin grants can still submit approvedLoanAmount and repayment schedule fields, so 0 < 1 required administrator checks protect credit terms that later feed borrower confirmation and existing unpaid loan schedules.

Recommendation

Add `@Roles('admin')` to mutation routes that alter application workflow or credit economics, and keep auditor access read-only or explicitly comment-only. Enforce the same authority check in `AdminApplicationsService.updateApplication` so controller metadata cannot be the only protection.

Techdollar: Resolved with [@dc786e7 ...](#)

Zenith: Verified.

[M-5] LoanOnchainRepaymentSyncService progress invariant failure past 200 loans

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan-onchain-repayment-sync.service.ts](#)
- [loan-overdue-check.service.ts](#)
- [admin-loans.service.ts](#)

Description

Every active on-chain loan must eventually enter the repayment event query while it can still be marked overdue. `LoanOnchainRepaymentSyncService.runOnce()` selects only take: 200 loans with no cursor or rotation, and I traced the downstream path from `hashToLoan` to `LoanOverdueCheckService.runOnce()` where unpaid backend state can become defaulted (the on-chain payment is irrelevant if never indexed here). With 256 active on-chain loans, $256 > 200$ leaves at least 56 loans outside each stable first page and their repayment events never reach `recordIndexedOnchainPrincipalRepayment()`. Paid borrowers can remain delinquent in the backend and later satisfy the admin liquidation precondition solely because state advanced to defaulted.

```
const activeLoans = await this.loanRepository.find({
  where: { status: In([...activeLoanStatuses]), isOnchain: true },
  take: MAX_ACTIVE_LOANS_PER_BATCH,
});

if (activeLoans.length === 0) {
  return;
}
```

Recommendation

Replace the fixed first page with deterministic pagination, a persisted cursor, or ordered rotation that guarantees coverage of all active on-chain loans. The overdue/default job should also account for unindexed repayment uncertainty before allowing liquidation-critical state changes.

Techdollar: Resolved with [@be04572 ...](#)

Zenith: Verified.

[M-6] KYC state regression through stale User persistence

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [sumsub.service.ts](#)
- [kyc.guard.ts](#)

Description

A later terminal KYC rejection must not be overwritten by an older verified entity held by an enrichment task. I traced `handleWebhook` through `persistStatus` and then into `syncApplicantInfo`; the green review mutates the same `User` object to `verified`, launches a delayed `applicant-info` fetch, and later saves that stale object after touching only profile fields (a classic lost update). The guard trusts the local row.

```
if (lastName && lastName !== user.lastName) {  
  user.lastName = lastName;  
  changed = true;  
}  
if (kycEmail && kycEmail !== user.kycEmail) {  
  user.kycEmail = kycEmail.toLowerCase();  
  changed = true;  
}
```

If webhook A writes `verified` at time 256 and webhook B writes `rejected` at time 512, a delayed save from A can still make `512 > 256` irrelevant to the database ordering that matters. That restores access to `KycGuard`-protected flows for a user `Sumsub` has already rejected.

Recommendation

Persist applicant enrichment with a column-limited update that cannot write `kycStatus` or `kycCompletedAt`. Alternatively reload the row in a transaction and refuse to save enrichment when the current KYC state is no longer verified for the same applicant.

Techdollar: Resolved with [@bb8eef5 ...](#)

Zenith: Verified.

[M-7] EquityApplicationsService upload path erases approval state

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [equity-applications.service.ts](#)

Description

An approved application should not be demoted by a borrower document upload, yet addDocument denies only confirmed and rejected before forcing any other non-submitted state back to submitted. I traced EquityController.addDocument -> EquityApplicationsService.addDocument; the state write runs after the file upload and does not distinguish approved from earlier review states (the branch is too broad).

```
const application = await this.queryService.findBareByIdForUser(id,
  user.id);
if (!application) return { success: false, error: 'Application not
  found' };
if (['confirmed', 'rejected'].includes(application.status)) return {
  success: false, error: 'Cannot add documents to this application' };

const filestore = await this.filestoreService.uploadFile(file, user.id, id);
if (application.status !== 'submitted') {
  application.status = 'submitted';
  await this.equityApplicationRepository.save(application);
```

For one approved application and one accepted file, 1 + 1 = 2 inputs are enough to remove the admin-approved transition. The borrower cannot then confirm, because updateApplication only confirms from approved, so the workflow waits for a manual review repair.

Recommendation

Change the upload transition to preserve terminal and post-review states, or allow uploads only while the application is still draft/submitted/action-required according to the intended state machine.

Techdollar: Resolved with [@038dfb8 ...](#)

Zenith: Verified.

[M-8] Borrower loan endpoint leaks admin internal fields

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loans-read.service.ts](#)
- [loan.mapper.ts](#)

Description:

The `getLoan` loads

```
relations:
  ['equityApplication', 'equityApplication.company', 'payments', 'schedule']
```

with no `select` projection, and `mapBorrowerLoan` returns `{ ...loan }`, spreading the fully hydrated child entities into the JSON response. No entity marks these columns `select: false`, and there is no `ClassSerializerInterceptor` or `@Exclude` anywhere in the codebase, so they serialize verbatim. The sibling `getLoans` list endpoint does restrict the same relations with an explicit `select`, proving the omission is unintended.

The leaked fields are: on `EquityApplication` - `reviewNotes` (the admin's private underwriting notes, written directly from admin `adminNotes` input at `admin-applications.service.ts:161`), `collateralCustodyDetails`, `transferAgentInfo`, `boardApprovalStatus`; on the nested `Company` - internal notes and `approvedBy`.

On `Payment` - notes (admin remarks), `failureReason`, `payerId`, `walletAddress`. The same missing-projection root cause affects `getPayments`. Any authenticated borrower can call, for example, `GET /loans/{ownLoanId}` and reads `data.equityApplication.reviewNotes`, `data.equityApplication.company.notes`, `data.payments[].notes`, etc. It is scoped to the borrower's own loan (not cross-tenant), but a loan only exists post-review so the sensitive fields are realistically populated.

Recommendations:

Add explicit `select` projections to the nested relations in `getLoan` and `getPayments`, mirroring the equity module's existing borrower allow-lists, or map the loan through a whitelisted borrower DTO instead of spreading the raw entity. As defence-in-depth, adopt a global serialization allow-list, so `ClassSerializerInterceptor` + `@Exclude` on sensitive columns.

Techdollar: Resolved with [@2c4eab2 ...](#)

Zenith: Verified.

[M-9] Onchain authorization registered with the uncapped amount

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [equity-applications.service.ts](#)
- [loan.service.ts](#)

Description:

When a borrower confirms an approved application through PUT /equity/applications/:id with status: 'confirmed', the confirmApprovedApplication derives a local `loanAmount = application.approvedLoanAmount ?? application.requestedLoanAmount`.

This value is passed to `createLoanOffer`, which correctly caps the persisted `loan.loanAmount` to the collateral maximum: `Math.min(options.loanAmount, calculateMaxLoan(collateralValue, maxLtv))`. But the next statement registers the original, un-capped local variable onchain:

```
await this.ethereumService.registerLoanVerification(borrowerWallet,
  loanAmount, loan.id);
```

The `registerLoanVerification` scales this to 6 decimals and calls `registerLoan(borrower, amountScaled, loanId)` unconditionally. The `requestedLoanAmount` is user controlled and validated only as "positive" as there is no LTV ceiling anywhere in the create or update path, and the admin approved transition performs no amount computation, so `approvedLoanAmount` stays null and the user's figure is used verbatim. Potential scenario is that borrower requests 1,000,000 against collateral supporting \$5,000, then admin routinely approves (the DB row still shows the correctly capped \$5,000, giving false confidence), borrower confirms, the oracle is authorised for \$1,000,000 while the DB loan is \$5,000.

Recommendations:

Register the persisted, capped value. Pass `parseFloat(String(loan.loanAmount))` to `registerLoanVerification`, never the pre-cap local variable. Independently, enforce the LTV ceiling by rejecting over-limit requests rather than silently `Math.min`, so the database, oracle, and business intent cannot diverge.

Techdollar: Resolved with [@ff38718 ...](#)

Zenith: Verified.

[M-10] Turnkey session JWT not bound to the expected organization

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- turnkey-auth.guard.ts

Description:

The application entire authentication boundary is enforced only by "was this token signed by Turnkey," not "does this token belong to our Turnkey organisation." An outsider who can obtain any Turnkey-signed session JWT can authenticate as a first class user and reach every user tier workflow. The `TurnkeyAuthGuard.canActivate` authenticates a request in three steps:

1. The `verifySessionJwtSignature(sessionJwt)` which, per the official `@turnkey/crypto` implementation, verifies the JWT against Turnkey's global production notarizer key and takes no organization parameter
2. An expiry check
3. The `userService.getOrCreateUser(decoded.userId, decoded.organizationId)`. The guard never compares `decoded.organizationId` against the configured `TURNKEY_DEFAULT_ORGANIZATION_ID`, and never validates an audience, issuer or session_type claim. The `turnkey.defaultOrganizationId` is referenced only when constructing the SDK client, never in the guard.

Because Turnkey is a multi-tenant custody service where organisations and sub-organisations can be created freely, a session JWT minted for an attacker's own Turnkey org, or issued by any other Turnkey-integrated application, satisfies `verifySessionJwtSignature`. The `getOrCreateUser` then provisions a fresh `role: 'user', isActive: true` record and the request is authenticated.

Recommendations:

After decoding, reject the request unless `decoded.organizationId` is `TURNKEY_DEFAULT_ORGANIZATION_ID` or is a verified sub-organisation of it (maintain an

allowlist of accepted org IDs). Additionally validate the `session_type`, and audience and issuer claims if Turnkey provides them.

Techdollar: Resolved with [@902928b ...](#)

Zenith: Verified.

[M-11] UR_TESTNET_URL and SUBGRAPH_URL accept plaintext authority endpoints

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [env.validation.ts](#)
- [subgraph-client.service.ts](#)
- [ur-api.service.ts](#)

Description

Sensitive outbound endpoints should fail closed unless their scheme is HTTPS, but configuration validation accepts generic URIs for RPC, subgraph, and UR banking URLs. I traced `env.validation` -> `SubgraphClientService` and `UrApiService`; the accepted URL is passed directly into `HttpLink` or `axios.create`, and headers carrying API keys or request signatures follow that scheme (configuration becomes the transport policy). For a configured `http://` UR base URL, 300 > 0 seconds of `X-API-Deadline` validity still leaves signed onboarding requests observable on plaintext transport. Subgraph credentials and onboarding PII can therefore cross an attacker-visible channel when deployment configuration is downgraded or mis-set.

```
// Ethereum
ETH_RPC_URL: Joi.string().uri().required(),
ETH_CHAIN_ID: Joi.number().integer().required(),
ADMIN_WALLET_PRIVATE_KEY: Joi.string()
  .pattern(/^(0x)?[0-9a-fA-F]{64}$/)
  .required(),

// Subgraph
SUBGRAPH_URL: Joi.string().uri().required(),
```

Recommendation

Require `https` in validation for `ETH_RPC_URL`, `SUBGRAPH_URL`, `UR_TESTNET_URL`, and `UR_MAINNET_URL`, and assert the parsed scheme again before constructing long-lived outbound clients.

Techdollar: Resolved with [PR-22](#).

Zenith: Verified.

[M-12] Sumsb action tokens use the provider's internal ID as externalActionId

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [sumsub.service.ts](#)
- [user.entity.ts](#)

Description:

The `createApplicantAction` generates an `externalActionId` and submits it to Sumsb. Sumsb returns its internal id together with the original external identifier, but the service stores the internal `result.id` as `urActionId`.

The `getActionAccessToken` later sends this internal ID in the field named `externalActionId`.

Sumsb requires the application-controlled external identifier when requesting an action token. The current request does not identify the action using the value required by the documented API, so the platform cannot reliably request a token bound to the intended UR verification action. This can prevent a user from completing the additional verification process.

Recommendations:

Store the internal Sumsb action ID and the application controlled external action ID in separate fields. Use the original external identifier when generating the action token and add an integration test that verifies identifier continuity across both requests.

Techdollar: Resolved with [@faeb8de...](#)

Zenith: Verified.

[M-13] Explicit 0% interest overrides fall back to the company or default rate

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan.service.ts](#)
- [admin-loans.service.ts](#)

Description:

The validation schema permits an interest rate of zero, and the application workflow preserves zero as an explicitly approved value. Loan creation later selects the interest rate using the following truthiness-based fallback:

```
options.interestRatePercent || company.interestRatePercent || 10
```

Because zero is treated as false, an approved 0% override is ignored.

The persisted rate is later used when constructing the on-chain loan terms. A borrower approved for a zero-interest loan can therefore receive a loan that charges the company rate or the 10% default.

Recommendations:

Use nullish fallback logic:

```
options.interestRatePercent ??  
company.interestRatePercent ??  
10
```

Techdollar: Resolved with [@b0567d0 ...](#)

Zenith: Verified.

[M-14] Public webhook callers share a rate-limit bucket behind the ALB

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [main.ts](#)
- [app.module.ts](#)

Description:

The application is deployed behind an internet-facing Application Load Balancer, but Express proxy trust is not configured. As a result, `req.ip` identifies the ALB connection rather than the original external caller.

The throttling guard uses `req.ip` to identify callers. Distinct clients reaching the same ECS task are therefore placed in the same rate-limit bucket. Webhook signature verification happens after this guard, so invalid requests consume the same capacity used by legitimate Sumsu and UR callbacks.

An unauthenticated attacker can send approximately 100 invalid requests per minute to each active task and cause valid provider callbacks to receive HTTP 429 responses. Sustaining the traffic can delay KYC and UR onboarding.

Recommendations:

Configure Express to trust only the known ALB hop or trusted proxy range. Add source-based limits at the edge and reserve separate rate-limit capacity for authenticated provider webhooks.

Techdollar: Resolved with [@24774f7 ...](#)

Zenith: Verified.

[M-15] addApplicationComment bounded-write validation gap

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [add-application-comment.dto.ts](#)
- [equity-applications.service.ts](#)
- [equity-application.entity.ts](#)

Description

An application comment thread should have bounded per-record growth, but comments is an inline jsonb array and borrower input has no maximum length or count. I traced addApplicationComment() through findBareByIdForUser() and the detail refetch; each accepted message appends to the prior array, saves the whole EquityApplication, then reloads a profile that includes comments (persistent cost, not transient parsing).

```
const application = await this.queryService.findBareByIdForUser(id,
  user.id);
if (!application) return { success: false, error: 'Application not
  found' };

application.comments = [ ... (Array.isArray(application.comments)
  ? application.comments : []), { who: 'user', timestamp: Date.now(),
  message } ];
await this.equityApplicationRepository.save(application);

const updatedApplication = await
  this.queryService.findByIdForUser(application.id, user.id,
  'BORROWER_DETAIL');
```

The row grows monotonically. A borrower can add $256 + 256 = 512$ large comments over time, causing later borrower reads, admin detail reads, and admin comment writes to carry the inflated JSON value until manual cleanup.

Recommendation

Move comments to a separate paginated table or enforce strict message length and comment-count limits before appending. Keep detail selectors from loading the full thread unless the caller explicitly requests a bounded page.

Techdollar: Resolved with [@c914356 ...](#)

Zenith: Verified.

[M-16] getPagination amplification through unchecked page

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [common.schemas.ts](#)
- [helpers.ts](#)
- [upcoming-loans-query.dto.ts](#)

Description

Pagination validation should bound both response size and database offset, but shared and upcoming-loan schemas cap only `limit` while accepting any integer `page` ≥ 1 . I traced `paginationSchema` and `getUpcomingLoansQuerySchema` into `getPagination`; the helper multiplies unchecked `page` into `offset` and services pass that value to TypeORM `skip` (the response cap is not a work cap).

```
export const getPagination = (page = 1, limit = 20) => {
  const offset = (parseInt(String(page)) - 1) * parseInt(String(limit));
  return {
    limit: parseInt(String(limit)),
    offset,
  };
};
```

For `page=1024` and `limit=100`, the database receives $(1024 - 1) * 100 = 102300$ skipped rows before returning at most 100. Blog, recent-activity, and upcoming-loan listings therefore expose caller-controlled scan/discard work as tables grow.

Recommendation

Set a maximum `page` or maximum derived `offset` for every offset-paginated schema, or migrate these listings to cursor pagination where large skips are not representable.

Techdollar: Resolved with [@9752b9e...](#)

Zenith: Verified.

[M-17] UpcomingLoansService projection invariant exposes raw Company

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [upcoming-loans.service.ts](#)
- [upcoming-loans.controller.ts](#)
- [company.entity.ts](#)
- [companies.service.ts](#)

Description

A public upcoming-loan response must serialize only a public-safe company projection. `UpcomingLoansService.findAll()` and `findOne()` hydrate `relations: ['company']`, and the `@Public()` controller returns those entities directly; I traced the `Company` entity and public company service, where `notes`, `transferRestrictions`, `approvedBy`, and `visibility` flags are present in the entity but omitted from catalog projections (the endpoint skips that mapper).

```
const [rows, count] = await this.upcomingLoanRepository.findAndCount({
  relations: ['company'],
  order: { createdAt: 'DESC' },
  take: limit,
  skip: offset,
});
return { rows, count };
```

One upcoming loan can expose 4 internal company fields where the public catalog exposes 0 of those fields, so $4 > 0$ crosses the intended API boundary. Hidden or inactive company metadata can also remain visible because the relation query lacks the `approved/active` predicate used by `/companies`.

Recommendation

Return an explicit DTO projection for nested company data and apply `approved/active` visibility filters on public upcoming-loan reads. Do not serialize raw TypeORM relations from public controllers.

Techdollar: Resolved with [PR-23](#).

Zenith: Verified.

[M-18] walletAddress canonicalization invariant missing from linkWallet

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [common.schemas.ts](#)
- [user.service.ts](#)
- [common-info.service.ts](#)

Description

One EVM address should map to one platform wallet identity, but validation preserves caller casing and `linkWallet` queries the unique column by exact string. I traced `evmAddressSchema -> UserService.linkWallet -> referral aggregation; storage` treats case variants as different values, while `CommonInfoService` later lowercases wallets before bonus calculation (identity collapses after duplication).

```
async linkWallet(user: User, walletAddress: string) {
  const existingUser = await this.userRepository.findOne({
    where: { walletAddress },
    select: ['id'],
  });

  if (existingUser && existingUser.id !== user.id) {
    throw new ConflictException('Wallet address already linked to another account');
  }
}
```

Two accounts can store lowercase and uppercase encodings of the same address, so $1 + 1 = 2$ accepted rows for one wallet identity. Referral logic then lowercases invitee wallets and adds each entry, allowing duplicate case-derived accounts to inflate points for a single underlying wallet.

Recommendation

Canonicalize EVM addresses before uniqueness checks and persistence, then apply the same normalized representation to referral lookup, aggregation, and API queries.

Techdollar: Resolved with [@ffc2b34 ...](#)

Zenith: Verified.

[M-19] On-chain repayment sync stops ingesting after the first 100 subgraph events

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan-onchain-repayment-sync.service.ts](#)
- [loan-overdue-check.service.ts](#)

Description:

The ONCHAIN_REPAYMENT_EVENTS_QUERY filters only on `loanId_in` and omits the first argument, so graph-node applies its default page size of 100 and `orderDirection: asc` returns the oldest events. Deduplication is performed client-side inside `runOnce` through the `evmEventId` lookup, which means events that already produced a `Payment` row still consume slots in that 100-row window. Once the active on-chain loans in a batch hold more than 100 lifetime repayment events between them, newer events fall outside the page and `recordIndexedOnchainPrincipalRepayment` is never invoked for them. Because this sync is the only writer of `evmEventId`, the loan retains a stale `nextPaymentDue` and `checkOverdueLoans` then sets `loan.status` to `defaulted` with a false `defaultReason` for a borrower who provably repaid on-chain.

The same codebase already paginates correctly in `common-info.service.ts` using `first` and `skip`, which indicates an oversight rather than a deliberate design choice.

```
// loan-onchain-repayment-sync.service.ts:57
onchainLoanRepaymentEvents(where: { loanId_in: $loanIds }, orderBy:
  timestamp, orderDirection: asc) {
```

Recommendations:

Add an explicit `first` page size and iterate with a cursor on `id_gt` or `timestamp_gt` until the result set is exhausted, and narrow the `where` clause so already ingested events are not re-fetched. Alarm when a page returns exactly the page size, because the onset of this failure is otherwise completely silent.

Techdollar: Resolved with [@be04572 ...](#)

Zenith: Verified.

[M-20] getTVL reserve-direction invariant failure in header accounting

SEVERITY: Medium

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: High.

Target

- [blockchain.action.ts](#)
- [blockchain.query.ts](#)
- [Stats.tsx](#)

Description

A header value labeled TVL must derive from an authoritative protocol-backing quantity, but this one does not. I traced Stats through BlockchainQuery.GetTVL() to getTVL(); the terminal read asks the USDTE ERC-20 for balanceOf(Contracts.AUSD.address), making USDTE the asset and AUSD the holder (the reserve direction is inverted). The result has no reserve semantics.

```
export async function getTVL(): Promise<BigNumberish> {
  try {
    const ausdBalance = await readContract<bigint>(Contracts.USDTE,
      'balanceOf', [Contracts.AUSD.address])
    return formatBigNumberish(ausdBalance)
  } catch (error) {
    console.error('Error getting TVL:', error)
    return EMPTY_BIG_NUMBERISH
  }
}
```

At Ethereum block 25,522,827, the queried USDTE.balanceOf(AUSD) was zero; both USDTE.getAUSDBalance() and AUSD.balanceOf(USDTE) returned 1.503204, while eUSDte totalBacking was 1,101,121.999998. The comparison $1.503204 > 0$ proves that the displayed source diverged from reserve-facing readings in the same snapshot. A direct USDTE transfer could move the header number, but it costs the sender those tokens and leaves protocol accounting unchanged. Users consequently receive a materially false health indicator whenever the header stat is displayed.

Recommendation

Define the header's TVL basis, then make `getTVL()` read that authoritative contract metric rather than `USDTE.balanceOf(AUSD)`; `getAUSDBalance()` is the existing reserve accessor when AUSD reserves are intended. Add a regression test that compares the displayed value against the selected accounting source.

Techdollar: Resolved with [PR-45](#).

Zenith: Verified.

[M-21] Uncursored take semantics block newer loan activation

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan-initiation-sync.service.ts](#)
- [ur-onboarding-sync.service.ts](#)
- [ur-onboarding.service.ts](#)

Description

A capped background poller must prove progress for records outside the current batch. `LoanInitiationSyncService.runOnce()` reads take: 200 pending loans without order, and `UrOnboardingSyncService` reads take: 10 accounts in sticky onboarding states; I traced `pollKycCheck()` and found `identity_verification` rows can remain unchanged while occupying the same selection window (there is no aging rule). If 16 UR accounts are stuck and the batch size is 10, then $16 > 10$ leaves later accounts unreachable under a stable database return order. New loans or onboarding records can remain pending indefinitely even after their external state is ready to advance.

```
const accounts = await this.urAccountRepository.find({
  where: {
    onboardingState: In(['creating_account', 'identity_verification']),
  },
  take: MAX_PER_BATCH,
});

if (accounts.length === 0) return;
```

Recommendation

Add deterministic ordering plus cursor/rotation semantics, or mark stale pending rows failed/deprioritized after a bounded retry window. Apply the same progress rule to loan initiation and UR onboarding pollers.

Techdollar: Resolved with [@d8cee93...](#)

Zenith: Verified.

[M-22] allocateInterestPayment state writes precede Payment uniqueness

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan.service.ts](#)
- [payment.entity.ts](#)

Description

An indexed EVM repayment should produce exactly one atomic state transition across schedules, loan totals, and the receipt row. `recordIndexedOnchainInterestRepayment()` checks `evmEventId`, calls `allocateInterestPayment()`, and only then saves `Payment`; I traced `allocateInterestPayment()` and it persists schedule rows plus `loan.totalInterestPaid` before the unique payment row can reject a duplicate.

```
loan.unpaidInterest = round6(unpaidInterest);
loan.totalInterestPaid = round6(toNum(loan.totalInterestPaid, 0)
    + interestPaid);

if (rowsToSave.length > 0) {
    await this.scheduleRepository.save(rowsToSave);
}

await this.loanRepository.save(loan);
```

Two workers handling the same event can each apply 256 interest before one `Payment` insert fails, so accounting observes $256 + 256 = 512$ credited from one chain event. The uniqueness constraint then protects only the ledger row, leaving earlier loan and schedule mutations durable or partially applied.

Recommendation

Move event-id check, allocation, loan/schedule mutation, and `Payment` creation into one database transaction with row locking or insert-first idempotency. The losing duplicate should roll back before touching loan accounting.

Techdollar: Resolved with [@eceb1fb ...](#)

Zenith: Verified.

[M-23] MAX_FILE_SIZE post-parse validation gap

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [equity.controller.ts](#)
- [upload-limit.guard.ts](#)
- [filestore.service.ts](#)

Description

Multipart body size must be rejected while the parser is consuming the stream, but addDocument installs FileInterceptor('file') without a Multer limits.fileSize cap and checks MAX_FILE_SIZE only after @UploadedFile() exists. I traced UploadLimitGuard before the interceptor and FilestoreService.uploadFile() after it; the guard trusts content-length, and storage consumes file.buffer, proving the request has already been materialized in memory (chunked requests bypass the estimate).

```
const totalSize = files.reduce((acc, file) => acc + Number(file.size || 0), 0);
const contentLength = parseInt(request.headers['content-length'] || '0', 10);

if (totalSize + contentLength > MAX_USER_LIMIT) {
  throw new ForbiddenException(
    `Upload limit exceeded. Maximum allowed storage:
    ${MAX_USER_LIMIT / (1024 * 1024)}MB. Current usage:
    ${Math.round(totalSize / (1024 * 1024))}MB`,
  );
}
```

The cap arrives late. A request declaring 0 < 5242880 bytes can stream far more data without tripping the header-based guard, forcing parsing and buffer allocation before the validator rejects or the worker runs out of capacity.

Recommendation

Configure `FileInterceptor` with `Multer limits.fileSize` equal to `MAX_FILE_SIZE` and reject missing or inconsistent length metadata before parsing when possible. Keep the existing validator as defense in depth, not as the first byte limit.

Techdollar: Resolved with [@3fdcd96...](#)

Zenith: Verified.

[M-24] loan-onchain-repayment-sync status-gated recovery flaw

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [loan-overdue-check.service.ts](#)
- [loan-onchain-repayment-sync.service.ts](#)
- [loan.service.ts](#)

Description

On-chain repayment finality must be able to repair backend loan state, but the overdue job can mark a loan defaulted before the repayment sync indexes a valid chain event. I traced `LoanOverdueCheckService.runOnce()` into `loan-onchain-repayment-sync.service.ts` and `recordOnchainPrincipalRepayment()`; after defaulting, the loan is outside `activeLoanStatuses`, so the poller no longer queries it and ingestion rejects it if reached directly (the database status becomes a one-way gate). Recovery is excluded. If an on-chain payment covers `1024 > 0` due amount before the grace cutoff but the subgraph indexes it after defaulting, backend state remains defaulted and can satisfy liquidation preconditions despite the borrower having paid.

```
const loan = await this.loanRepository.findOne({ where: { id: loanId },
  relations: ['borrower', 'equityApplication'] });
if (!loan) {
  throw new Error('Loan not found');
}
if (!(activeLoanStatuses as readonly string[]).includes(loan.status)) {
  throw new Error(`Cannot record on-chain principal repayment for loan in
    status: ${loan.status}`);
}
```

Recommendation

Reconcile on-chain repayments before marking a loan defaulted and allow late-indexed repayment ingestion to reopen or settle a stale default when the event predates liquidation. Liquidation should require a fresh repayment check, not only the stored defaulted status.

Techdollar: Resolved with [@4bb201c ...](#)

Zenith: Verified.

[M-25] Cron exclusivity invariant failure in runExclusive

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [app.module.ts](#)
- [run-exclusive.ts](#)
- [loan-overdue-check.service.ts](#)
- [ur-onboarding-sync.service.ts](#)

Description

Scheduled state transitions require exclusivity across replicas, not only inside one Node process. I traced `ScheduleModule.forRoot()` and the cron services; every application instance starts schedulers, while `runExclusive` and the UR sync service store booleans in memory, so each replica can select the same rows and call the same external endpoints at the same time. The deployment docs acknowledge multi-replica operation. With 4 replicas, a batch cap of 10 UR accounts can produce $40 > 10$ provider polls for the same scheduler tick. The same coordination defect applies to loan jobs that read and mutate financial state without row claims or a distributed lease.

```
export async function runExclusive(guard: { isRunning: boolean } | any,
  logger: Logger, jobName: string, fn: () => Promise<void>):
  Promise<void> {
  if (guard.isRunning) {
    logger.debug(`${jobName}: previous run still in progress,
  skipping`);
    return;
  }
  guard.isRunning = true;
  try {
```

Recommendation

Run cron workers behind a database-backed lease or deploy them as a single designated worker process. For batch jobs, claim rows transactionally before processing so multiple replicas cannot own the same work item.

Techdollar: Resolved with [@3a8f8fb ...](#)

Zenith: Verified.

4.3 Low Risk

A total of 36 low risk findings were identified.

[L-1] Concurrent create-account calls orphan UR accounts

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [ur-onboarding.service.ts](#)
- [ur-account.entity.ts](#)
- [ur.controller.ts](#)

Description

UR onboarding should claim a user's local account slot before any external account is provisioned. `start()` first queries `urAccountRepository.findOne({ userId })`, then calls `/api/fma/v1/create-account`, then saves `UrAccount`; I traced the entity and the unique `user_id` constraint fires only after the provider call has already returned (too late).

```
const result = await this.urApi.post<CreateAccountData>(  
  '/api/fma/v1/create-account',  
  {  
    applicantId,  
    email,  
  },  
  {  
    externalUserId: user.id,  
  },  
);
```

Two concurrent `start` requests can produce $2 > 1$ provider accounts for the same user, while only one `ur_accounts.user_id` row may persist. The losing request can surface as an error, and the extra UR-side account remains unreferenced by `User.urId` or local onboarding state.

Recommendation

Serialize onboarding start per user with a transaction, lock, or insert-first pending row before calling UR. On unique-constraint conflict, reload the existing UrAccount and return its state instead of allowing a second provider account to be orphaned.

Techdollar: Resolved with [PR-20](#).

Zenith: Verified.

[L-2] confirmApprovedApplication uniqueness invariant missing database enforcement

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [equity-applications.service.ts](#)
- [loan.service.ts](#)
- [loan.entity.ts](#)

Description

Each equity application should have at most one loan, but the confirmation path checks `Loan.equityApplicationId` without a row lock or a database uniqueness constraint. I traced `EquityApplicationsService.confirmApprovedApplication -> LoanService.createLoanOffer`; the current code does use a transaction, so the original no-transaction claim is stale, but the transaction does not serialize competing reads for the same application row.

```
let loan = await manager.findOne(Loan, { where: { equityApplicationId: application.id } });
if (!loan) {
  const termMonths = application.termMonths ?? 12;
  if (application.termMonths === null) {
    application.termMonths = termMonths;
    application.numberOfPayments = termMonths;
  }
  await manager.save(EquityApplication, application);
}
```

Two concurrent confirmations can both observe zero loans, so $0 + 2 = 2$ inserts remain possible when the database accepts non-unique `equity_application_id`. Separate loan IDs can then progress through collateral and disbursement flows against the same application.

Recommendation

Add a unique constraint on `loans.equity_application_id` and serialize confirmation on the application row, handling unique-conflict retries as idempotent success when the existing loan belongs to the same application.

Techdollar: Resolved with [@1802e1d...](#)

Zenith: Verified.

[L-3] buildDbSslConfig transport-security invariant failure

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium.

Target

- [db-ssl.ts](#)
- [env.validation.ts](#)
- [database.module.ts](#)
- [ormconfig.ts](#)

Description

Database connections must authenticate the PostgreSQL server in production, but `buildDb-sslConfig` returns plaintext mode when `DB_SSL` is absent and unauthenticated TLS when the CA bundle is missing. I traced the helper into both `database.module.ts` and `ormconfig.ts`; runtime connections and migrations receive the same `ssl` value, while `env.validation.ts` defaults `DB_SSL` to `false` and never couples `DB_SSL_CA_PATH` to certificate verification (fail-open by configuration).

```
const ca = resolveCaCertificate();
if (ca) {
    return { rejectUnauthorized: true, ca };
}

logger.warn(
    'DB_SSL is enabled but no CA certificate was found (set DB_SSL_CA_PATH ' +
    'or ship certs/rds-global-bundle.pem). Falling back to an UNVERIFIED ' +
    'TLS connection ' +
```

The channel is not authenticated. With 1 > 0 missing CA files, the process still starts and accepts an unverified database peer, so credentials, loan records, user data, and migration reads can be intercepted or modified by a network-positioned attacker.

Recommendation

Require verified TLS for non-local database connections and throw during startup when the CA cannot be loaded. Remove the `rejectUnauthorized: false` fallback and make environment validation reject insecure combinations before TypeORM initializes.

Techdollar: Resolved with [@565cc8e ...](#)

Zenith: Verified.

[L-4] EVM settlement state transition without amount equality

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan-payment-confirmation-sync.service.ts](#)
- [payment.entity.ts](#)

Description

Queued banking payments should only be marked EVM-confirmed when the indexed on-chain event can be tied back to the expected settlement. The normal confirmation path does compare the indexed event owner against the loan's recorded disbursement multisig owner, so this is not a blanket "owner is never checked" issue:

- [loan-payment-confirmation-sync.service.ts#L150-L166](#)

However, when the expected owner is missing, the verifier accepts the event without that owner comparison:

- [loan-payment-confirmation-sync.service.ts#L208-L218](#)

The subsequent match is still broad. Payments are grouped by loan hash, events are processed chronologically, and the first unconfirmed payment of the matching broad kind is marked `confirmedOnEvm`. The code does not compare the queued `safeTxHash`, exact amount, or exact settlement identifier before flipping the flag:

- [loan-payment-confirmation-sync.service.ts#L168-L178](#)

The practical impact is limited because `confirmedOnEvm` does not appear to directly release funds. Still, for older or incomplete loans without a recorded expected owner, an unrelated same-kind event for the same loan hash can confirm the wrong backend payment and corrupt the operational audit trail.

Recommendation

Do not accept confirmation events without an expected owner unless the payment can be matched through another strong identifier. Compare the observed owner, event kind, amount, and settlement identity before setting `confirmedOnEvm`. For legacy rows missing `disbursementWalletAddress`, require a backfill or leave the payment unconfirmed for man-

ual review.

Techdollar: Resolved with [@f582d71 ...](#)

Zenith: Verified.

[L-5] `ILike` literal-search invariant failure

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [companies.service.ts](#)
- [companies-query.dto.ts](#)

Description

Company search results should preserve the same visibility boundary everywhere company identifiers are returned. The public catalog path applies `approved: true` and `isActive: true` before searching companies, so `getCompanies` does not expose inactive or unapproved rows through its normal listing flow:

- [companies.service.ts#L24-L36](#)

The `checkCompany` path does not apply that same base filter before lookup. It searches `name` and `legalName` using caller-controlled `ILike(%...%)`, selects `approved / isActive`, and then returns `companyId` and `companyName` even when the matched company is not approved or not active:

- [companies.service.ts#L197-L210](#)
- [companies.service.ts#L222-L231](#)

As a result, an authenticated user can probe broad names or wildcard characters and learn that hidden company records exist, including their internal ID and display name. This is not SQL injection, but it is an inconsistent visibility check on a company-discovery endpoint.

Recommendation

Apply the same `approved: true` and `isActive: true` predicate in `checkCompany` before returning any company identifier or name. If the product needs to tell users that an unapproved company exists, return only a generic “not currently eligible” response without exposing the internal company ID or canonical name. Also escape LIKE metacharacters or use a literal search primitive so `%` and `_` are treated as user text rather than pattern operators.

Techdollar: Resolved with [PR-21](#).

Zenith: Verified.

[L-6] LoanVerifier registration consistency loss in confirmApprovedApplication

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [ethereum.service.ts](#)
- [equity-applications.service.ts](#)

Description

LoanVerifier registration failures are mostly caught before backend state advances because `registerLoanVerification` performs a pre-flight `estimateContractGas` using the same call parameters before broadcasting the transaction:

- [ethereum.service.ts#L227-L239](#)

The remaining issue is narrower. After the estimate succeeds, the code calls `writeContract` but does not wait for a successful transaction receipt before returning to the caller:

- [ethereum.service.ts#L241-L252](#)

`confirmApprovedApplication` calls this function inside the confirmation flow, so a non-throwing broadcast is enough for the application confirmation transaction to complete:

- [equity-applications.service.ts#L461-L488](#)

This is not a broad revert-handling issue, because gas estimation catches the common failure cases. The residual risk is that a transaction can be dropped, replaced, or fail after estimation, leaving backend state advanced while the LoanVerifier registration never finalizes on-chain.

Recommendation

Either wait for a successful receipt before treating the verifier registration as complete, or persist an intermediate registration-pending state and reconcile the final on-chain registration asynchronously. Failed or missing registrations should be retryable without relying on a prior broadcast as proof of completion.

Techdollar: Resolved with [@1802e1d ...](#)

Zenith: Verified.

[L-7] Open redirect on /kyc via unvalidated callbackUrl

SEVERITY: Low**IMPACT:** Low**STATUS:** Resolved**LIKELIHOOD:** Medium

Target

- [page.tsx](#)
- [routes.ts](#)

Description:

The const `callbackUrl = searchParams.get('callbackUrl') || ROUTES.dashboard.root` is passed directly to `router.push(callbackUrl)` inside `handleComplete` in `page.tsx:18-22`, which runs only when `KYCWidget` reports `idCheck.onApplicantSubmitted`. No validation is applied to `callbackUrl`.

A victim who followed a link like `.../kyc?callbackUrl=https://evil.com` or protocol-relative `//evil.com` is hard-navigated off-site to an attacker page that looks like a continuation of the onboarding flow, so it is a convincing credential or asset-phishing vector. The `/kyc` page is reachable pre-authentication, but the redirect fires only after a successful Sumsu applicant submission, not on a bare visit. That means that the victim must complete the KYC flow for the redirect to trigger, which raises the interaction bar.

Recommendations:

Validate `callbackUrl` before navigating and default to an internal route on any mismatch. Accept only same-origin relative paths - reject any value that does not start with exactly one `/` blocking `//host` and any value containing a scheme, or resolve `new URL(callbackUrl, window.location.origin)` and require `.origin === window.location.origin`. Otherwise fall back to `ROUTES.dashboard.root`.

Techdollar: Resolved with [@0dd6fe9...](#)

Zenith: Verified.

[L-8] Wallet address linkable without proof of ownership

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [user.service.ts](#)
- [user.controller.ts](#)

Description:

The PUT `/user/wallet` sets `user.walletAddress` to any well-formed EVM address after only a format check and a first-claim uniqueness check. There is no signature proving control of the private key. That address is later used as the loan disbursement recipient and the referral-attribution key. Since `wallet_address` is `UNIQUE`, pre claiming a victim's known or predictable address causes the victim's later attempt to link that specific address to throw `ConflictException`, but the victim remains free to link a different address they own, so the impact is bounded.

The PUT `/user/profile` similarly accepts arbitrary `urId` and `rainId` strings. The `urId` squatting vector is weaker however as it requires the attacker to correctly predict the integer `urId` UR will assign to the victim before the server writes it.

Recommendations:

Require proof-of-control before binding `walletAddress`. Verify an EIP-191 or EIP-712 signature over a server issued nonce, or bind the address to the Turnkey session's derived key. Remove `urId` and `rainId` from the user editable profile schema entirely.

Techdollar: Resolved with [PR-56](#).

Zenith: Verified.

[L-9] Month-end schedule generation creates overlapping payment periods

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [helpers.ts](#)
- [loan-overdue-check.service.ts](#)

Description:

The schedule helper advances payment dates using JavaScript's `Date.setMonth`. When the target month does not contain the original day, JavaScript moves the date into the following month rather than consistently using the final valid day.

For a loan activated on January 31, the produced periods will be created:

```
January 31 to March 3
March 3 to April 3
March 31 to May 1
```

The second and third periods overlap, and the payment dates no longer follow a consistent monthly anchor. This can produce confusing due dates, inaccurate maturity dates, and incorrect overdue timing. However, based on the review, it should not lead to any additional interest charges.

Recommendations:

Define a clear month-end scheduling policy and use a calendar safe month addition function that clamps dates to the final valid day or preserves an end-of-month anchor.

Techdollar: Resolved with [@8dcffca...](#)

Zenith: Verified.

[L-10] Concurrent distinct banking receipts lose loan accounting updates

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan.service.ts](#)
- [loan-payment-schedule.entity.ts](#)

Description:

The `adminConfirmInterestPayment` updates the payment, loan aggregate, and scheduled obligations inside a database transaction. However, it reads the loan and its schedule without row locking, optimistic versioning, or serialization for the affected loan. Two concurrent requests with different banking transaction IDs and Safe hashes can read the same initial state. Both receipts and ledger entries are created, but each request calculates and writes the aggregate state using a stale snapshot.

This issue can cause the platform to continue demanding interest that it has already received.

Recommendations:

Serialize payment allocation for each loan using a pessimistic row lock, transaction-level advisory lock, or serializable transaction with retries. Lock both the loan and the affected schedule rows before calculating the allocation.

Techdollar: Resolved with [@7018d5d ...](#)

Zenith: Verified.

[L-11] Historical settlement events can be reused to confirm later payments

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan-payment-confirmation-sync.service.ts](#)
- [payment.entity.ts](#)

Description:

The `LoanPaymentConfirmationSyncService` retrieves historical settlement events but removes their unique identifiers during normalization. Pending payments are then matched using the loan hash, payment type, owner, and FIFO ordering. The service does not record that an event was already consumed or bind the event to the payment's Safe transaction.

This can incorrectly set `confirmedOnEvm` and conceal missing settlement activity in administrative or reconciliation views. No direct funds movement or attacker-controlled state transition was identified, which is why this should be treated as a Low operational integrity issue.

Recommendations:

Preserve and atomically store the event identifier in the existing `evmEventId` field. Reject identifiers that were already consumed and strengthen matching with the payment amount, event timestamp, transaction identity, and Safe execution data where available.

Techdollar: Resolved with [@4e79c41 ...](#)

Zenith: Verified.

[L-12] getLoanAmountByCompany predicate mismatch on company visibility

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- [common-info.service.ts](#)
- [common-info.controller.ts](#)
- [companies.service.ts](#)

Description

Public loan analytics must preserve the same company visibility predicate as public company reads. `getLoanAmountByCompany()`, `getLoanAmountBySector()`, and `getSectorTotals()` join company but filter only loan status; I traced `getCompanies()` and `getCompany()` and found the expected `approved: true`, `isActive: true` boundary absent from these aggregate paths (one public surface contradicts another).

```
const baseQuery = this.loanRepository
  .createQueryBuilder('loan')
  .innerJoin('loan.equityApplication', 'app')
  .innerJoin('app.company', 'company')
  .where('loan.status IN (: ... statuses)', { statuses:
    ACTIVE_LOAN_STATUSES });

if (companyId) {
```

If 1 hidden company has 256 active-loan units and 0 catalog visibility, the public aggregate still returns `256 > 0` with `company.id`, `company.name`, and sector metadata. Hidden or delisted companies can be enumerated through metrics that should inherit the catalog predicate.

Recommendation

Apply `company.approved = TRUE` and `company.isActive = TRUE` to every public analytics query that exposes company-derived rows or totals. Reuse one helper predicate so `getActiveLoans()` cannot mix filtered rows with unfiltered sector totals.

Techdollar: Resolved with [@41a3787 ...](#)

Zenith: Verified.

[L-13] getPoints pagination work invariant failure

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [common-info.service.ts](#)
- [common-info.controller.ts](#)
- [common.schemas.ts](#)

Description

Public pagination should bound request cost, but `getPoints` performs whole-leaderboard computation before slicing the requested page. I traced the unauthenticated controller into `common-info.service.ts`; the service uses cached raw users, recomputes live totals, walks the referral map, sorts the complete result set, and only then applies `offset` and `limit` (pagination limits output, not work). The refresh path can also ingest every subgraph user with no hard maximum. For a request asking for 20 rows against 4096 indexed users, 4096 > 20 computations still execute. The endpoint is public. An attacker who inflates wallets through cheap point-generating actions can make each request consume CPU, memory, and subgraph-derived state proportional to the full population, degrading unrelated API traffic sharing the same Node process.

```
for (const [inviterWallet, inviteeWallets] of referralMap) {
  let bonus = 0;
  for (const inviteeWallet of inviteeWallets) {
    const inviteeTotal = baseTotalsByWallet.get(inviteeWallet) ?? 0;
    bonus += inviteeTotal * REFERRAL_BONUS_RATE;
  }
  if (bonus > 0) {
    referralBonusByWallet.set(inviterWallet, bonus);
  }
}
```

Recommendation

Cache a pre-ranked leaderboard snapshot or push ranking and pagination into the data source with explicit maximums. Add request-independent caps for user ingestion and referral expansion so unauthenticated calls cannot scale backend work with attacker-created accounts.

Techdollar: Resolved with [@f7523b3 ...](#)

Zenith: Verified.

[L-14] safeTxHash canonicalization invariant failure

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [common.schemas.ts](#)
- [admin-loans.service.ts](#)
- [loan.service.ts](#)

Description

Safe transaction identity must be stored in one canonical representation before uniqueness checks execute. I traced txHashSchema into distributeLoan, confirmInterestPayment, and principal confirmation; validation accepts a bytes32-looking value, but the custom Joi rule returns the original string, and the services compare exact text (case included). EVM hash bytes are not case-dependent.

```
export const bytes32Schema = Joi.string()
  .trim()
  .min(1)
  .custom((value, helpers) => {
    if (!isHash(value)) return helpers.error('string.bytes32');
    return value;
  })
```

If lower-case and upper-case forms denote the same bytes, then 1 + 1 = 2 accepted database identities for one Safe transaction. That defeats idempotency for loan disbursements and payment confirmations that should bind exactly once to a queued Safe action.

Recommendation

Normalize every hash at the validation boundary and repeat the same canonicalization before lookup and persistence. Back the application rule with lower-case or byte-level uniqueness in the database.

Techdollar: Resolved with [@f582d71 ...](#)

Zenith: Verified.

[L-15] `response.Body` pipe without source error containment

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [filestore.service.ts](#)

Description

Every readable stream handed to an HTTP response must have an error path before control returns to the event loop. I traced stream in `FilestoreService`: the handler sets response headers, pipes `response.Body`, and relies on a surrounding try block that only covers setup, not later 'error' emissions (the pipe call returns immediately). The omission is local.

```
if (response.Body) {
  (response.Body as Readable).pipe(res);
} else {
  res.status(404).send('File not found');
}
}
} catch (error) {
```

One authenticated download can start normally, then an S3 or socket failure can emit 1 error with 0 listeners, and Node treats $1 > 0$ missing handlers as an uncaught exception. A repeatable crash on document reads is sufficient to degrade the backend for unrelated users.

Recommendation

Attach source and destination error handlers before piping, and destroy or end the opposite side on failure. Prefer `pipeline()` so stream errors are joined into one explicit failure path.

Techdollar: Resolved with [@9b2a4c6 ...](#)

Zenith: Verified.

[L-16] handleAccountResult live account consistency regression

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [ur-webhook.service.ts](#)
- [ur-account.entity.ts](#)

Description

A live UR account should be terminal for negative onboarding webhooks, but `UrWebhookService` checks duplicate failure fields instead of checking whether the account is already live. I traced `handleAccountResult()` and `handleKycRetryRequired()` against `UrAccount`; there is no persisted event identifier, and the rejected branch updates `accountStatus` and `onboardingState` from any current state (late delivery is enough). State goes backward. With $1 + 1 = 2$ reordered account events, activation can first set live, then a later rejection can set failed, leaving a completed custody account in a backend state that normal onboarding does not repair.

```
const rejectReason = String(data.rejectReason || 'Account rejected');
if (urAccount.accountStatus === 'failed' && urAccount.rejectCode ===
    rejectCode && urAccount.rejectReason === rejectReason)
{
    this.logger.debug(`Skipping duplicate rejection webhook for urAccount
    ${urAccount.id}`);
    return;
}

await this.urAccountRepository.update(urAccount.id, {
    onboardingState: 'failed',
    accountStatus: 'failed',
```

Recommendation

Make live a terminal state for rejection and retry-required handlers unless an explicitly newer provider event proves otherwise. Store and compare UR event ids or monotonic

timestamps before mutating UrAccount state.

Techdollar: Resolved with [@6eddfb7 ...](#)

Zenith: Verified.

[L-17] process.env.DB_SSL raw-value validation mismatch

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [env.validation.ts](#)
- [db-ssl.ts](#)

Description

The accepted configuration value for database TLS should be the value used by the database module, but validation and consumption apply different boolean semantics. `env.validation.ts` accepts truthy boolean strings for `DB_SSL`; I traced the runtime decision into `buildDbSslConfig()`, where the raw environment variable is compared only to lowercase `true` (operator-provided variables remain raw). The result is silent plaintext transport under a schema-approved setting. For example, `1 > 0`, with one accepted `TRUE` value and zero enabled TLS branches. The guard is exact. An operator can pass startup validation with `DB_SSL=TRUE`, yet the consumer returns `false`, so PostgreSQL traffic carrying credentials, borrower records, and KYC data can traverse the network without TLS while deployment health checks still report success.

```
export function buildDbSslConfig(): DbSslConfig {
  if (process.env.DB_SSL !== 'true') {
    return false;
  }
  const ca = resolveCaCertificate();
  if (ca) {
    return { rejectUnauthorized: true, ca };
  }
}
```

Recommendation

Read the coerced value from `ConfigService` or normalize `process.env.DB_SSL` once before any consumer sees it. Reject ambiguous boolean spellings if the runtime only supports one canonical representation.

Techdollar: Resolved with [@13ae23b...](#)

Zenith: Verified.

[L-18] getDashboard cardinality invariant failure for borrower applications

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [user.controller.ts](#)
- [equity-application-query.service.ts](#)
- [equity-applications.service.ts](#)

Description

A dashboard read invariant should cap borrower-owned rows before relation loading and response serialization. `getDashboard` calls `EquityApplicationQueryService.findAll()` with only `where: { userId }`, then maps every returned application; I traced `createApplication()` and found no per-user application ceiling to bound that set (the list endpoint has pagination, this path does not). If a borrower accumulates 512 draft applications and $512 > 20$ normal page items, one dashboard request loads and serializes the full relation graph rather than a bounded page. Repeated requests become a user-controlled database and JSON work multiplier against shared API capacity.

```
async findAll(options: {
  where: FindOptionsWhere<EquityApplication>;
  profile: ApplicationProfile;
  order?: Record<string, 'ASC' | 'DESC'>;
}): Promise<EquityApplication[]> {
  return this.repository.find({
    where: options.where,
    relations: RELATIONS_MAP[options.profile],
    select: PROFILE_MAP[options.profile]() as any,
```

Recommendation

Make `getDashboard` page or cap applications and `activeLoans`, using the same bounded query contract as the list endpoints. Add a borrower-level quota or archival rule if unlimited draft applications are not a product requirement.

Techdollar: Resolved with [@b0f7aee ...](#)

Zenith: Verified.

[L-19] evmAddressSchema recipient validation invariant omits zero-address rejection

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [common.schemas.ts](#)
- [user.service.ts](#)
- [admin-loans.service.ts](#)

Description

A borrower wallet recorded for disbursement should be a usable recipient, but the shared validator proves only EVM address syntax and returns the caller's exact value. I traced walletSchema -> linkWallet -> AdminLoansService.prepareLoanDistribution; after storage, the loan path tests only whether walletAddress is truthy (a weak state predicate). The zero address has 40 hex digits, so $40 > 0$ and the later non-empty check accepts it even though it is not a recipient the workflow should use. prepareLoanDistribution then saves safeTxHash, changes status to pending_disbursement_execution, and builds a Safe call with the unusable borrower address.

```
export const evmAddressSchema = Joi.string()
  .trim()
  .min(1)
  .custom((value, helpers) => {
    if (!isAddress(value)) return helpers.error('string.evmAddress');
    return value;
  })
  .messages({
```

Recommendation

Reject `0x0000000000000000000000000000000000000000000000000000000000000000` in wallet-recipient validation and reuse that stricter predicate before loan registration or Safe call generation.

Techdollar: Resolved with [@8940c07...](#)

Zenith: Verified.

[L-20] Turnkey and Sumsb identity data written to production info logs

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [turnkey.service.ts](#)
- [sumsub.service.ts](#)

Description:

The `turnkey.service.ts:88` logs the Turnkey email and userName at info level, and `sumsub.service.ts:301` logs firstName, lastName and kycEmail at info level. `LoggerModule` sets level to info whenever `NODE_ENV` is production or staging, so both statements are emitted in deployed environments. The redact configuration lists only `req.headers.authorization`, the `x-techdollar-api-key` header and `req.headers.cookie`, which are request header paths and do not apply to interpolated message strings. Anyone with read access to the centralized log sink therefore receives borrower identity data that is not required for debugging. No attacker capability is gained directly, so this is a data minimization and privacy exposure that widens the blast radius of any separate log access compromise.

```
// logger.module.ts:10-13
level: isProduction ? 'info' : 'debug',
redact: {
  paths: ['req.headers.authorization',
    'req.headers["x-techdollar-api-key"]', 'req.headers.cookie'],
  censor: '[REDACTED]',
```

Recommendations:

Replace the personal values in both log messages with internal identifiers such as `user.id` or the applicant id. Add message level redaction or structured fields for any case where correlation genuinely requires the underlying values.

Techdollar: Resolved with [@777de33 ...](#)

Zenith: Verified.

[L-21] Borrower can cancel an equity application after the loan has been created

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [equity-applications.service.ts](#)
- [admin-loans.service.ts](#)

Description:

The `cancelApplication` blocks only the rejected and confirmed statuses, so an application in `collateral_accepted` remains cancellable. That status is written by `createLoanOffer` in the same transaction that creates the `Loan` row and sets `loan.status` to `pending_disbursement`, and nothing advances the application past it, so `collateral_accepted` is precisely the window in which a loan already exists.

The deny list is inverted relative to lifecycle order, because it blocks the earlier confirmed state and permits the strictly later `collateral_accepted` state. Cancellation writes only `application.status` and `application.rejectionReason` without inspecting the loan, and `distributeLoan` gates on `loan.status` alone, so the loan still distributes against custodied collateral.

No collateral release path exists anywhere in the backend, so the terminal impact is an application record reading rejected while its loan funds and accrues.

```
// equity-applications.service.ts:347-350
if (['rejected', 'confirmed'].includes(application.status)) return {
  success: false, error: 'Cannot cancel application in current status' };

application.status = 'rejected';
application.rejectionReason = 'Cancelled by user';
```

Recommendations:

Replace the deny list in `cancelApplication` with an allow list of pre loan statuses and assert that no `Loan` row exists for the application. Apply the same from-status guard to `addDocument`, which carries the identical inverted list and force resets a live application back to submitted.

Techdollar: Resolved with [@91c6bad ...](#)

Zenith: Verified.

[L-22] Interest overpayment beyond the full remaining schedule is recorded but not allocated

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan.service.ts](#)
- [confirm-loan-payment.dto.ts](#)

Description:

The `confirmInterestPaymentSchema` validates `amount` with `Joi.number().positive()` and sets no upper bound, so `adminConfirmInterestPayment` accepts any positive value. The `allocateInterestPaymentTx` applies the payment to `loan.unpaidInterest` first, then to due rows and then to all future rows as advance interest, and its local remaining variable is discarded once every row is exhausted.

The Payment row therefore stores the full amount while `interestAmount` holds only `interestPaid`, which breaks the implicit relationship between `amount` and the sum of `interestAmount` and `principalAmount`. A residue only exists after the entire remaining lifetime interest of the loan has been covered, and no path other than `display` reads `Payment.amount`, so no balance, payoff quote or dashboard can be corrupted.

Both values are visible to both parties, because the confirming admin receives them in the same response and the borrower sees the same row in their payment history.

```
// loan.service.ts:439-440
amount: round6(amount),
interestAmount: round6(interestPaid),
```

Recommendations:

Reject an amount that exceeds the loan's total allocatable interest, or record the residue explicitly as an unapplied borrower credit so `amount` always equals the sum of its allocated parts.

Techdollar: Resolved with [@5c808b6 ...](#)

Zenith: Verified.

[L-23] S3 object key derived from deterministic inputs allows a same millisecond collision

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [filestore.service.ts](#)
- [filestore.entity.ts](#)

Description:

The `uploadFile` builds `hashInput` from `Date.now()`, `accountId` and `file.originalname`, then hashes it with SHA256 to produce key. The digest contributes no entropy because every input is deterministic, so two uploads by one account with the same filename in the same millisecond produce an identical key and one object overwrites the other. The `Filestore.key` carries no unique constraint, so both requests persist rows referencing a single object. Because `equityApplicationId` is passed into `uploadFile` but excluded from `hashInput`, a collision can span two different applications, and an admin reviewing application A can be served the bytes uploaded for application B. The catch block calls `deleteFileFromS3(key)` keyed on the shared key rather than on the row it created, so a database failure in the losing request removes the object the winning row still references.

```
// filestore.service.ts:178-181
const timestamp = Date.now();
const hashInput = `${timestamp}-${accountId}-${file.originalname}`;
const hash = crypto.createHash('sha256').update(hashInput).digest('hex');
const key = hash;
```

Recommendations:

Include `crypto.randomUUID()` in `hashInput` and add a unique constraint on `Filestore.key` with a matching migration. Scope the cleanup in the catch block so it deletes only an object the failing request actually created.

Techdollar: Resolved with [@3659add ...](#)

Zenith: Verified.

[L-24] Loan initiation sync activates a loan without checking the event type against `loan.isOnchain`

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loan-initiation-sync.service.ts](#)
- [loan.service.ts](#)

Description:

The `fetchInitiationEvents` reads `fiatLoanInitiatedEvents` and `onchainLoanInitiatedEvents` into one `results` array that retains only `loanIdHash`, `txHash`, `timestampSeconds` and `owner`, so the event type is discarded before matching occurs. `runOnce` then resolves the loan purely by hash, and `finalizeDisbursement` validates only `loan.status` without ever comparing the event type against `loan.isOnchain`. A `FiatLoanInitiated` event can therefore activate a loan whose `isOnchain` is true, leaving `disbursementTxHash` pointing at the wrong kind of transaction. The row keeps its original `isOnchain` value, so mode-gated consumers such as the repayment sync still route correctly and the terminal impact is one incorrect hash on one row. Reaching this state requires the fully trusted Safe multisig to execute a different function than the instruction string `distributeLoan` produced, because the backend never calls the contract itself.

```
// loan-initiation-sync.service.ts:128
const loan = hashToLoan.get(match.loanIdHash.toLowerCase());
```

Recommendations:

Preserve the event type in `fetchInitiationEvents` and compare it against `loan.isOnchain` in `runOnce`, then activate but raise an operator alert on mismatch rather than rejecting, because the funds have already moved on-chain. Do not write the event receiver into `disbursementWalletAddress`, because that field is by design the executing multisig owner and is reused as `expectedOwner` by the payment confirmation sync.

Techdollar: Resolved with [@96ceda5 ...](#)

Zenith: Verified.

[L-25] updateProfile authority invariant broken for custody identifiers

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [user.service.ts](#)
- [update-profile.dto.ts](#)
- [ur-onboarding.service.ts](#)

Description

UR account identifiers should be written by the UR onboarding integration, not by ordinary profile edits. The profile update DTO exposes `urId` as an optional client-controlled field:

- [update-profile.dto.ts#L4-L12](#)

`UserService.updateProfile` accepts that value and writes it to `user.urId` after only checking whether another local user already has the same identifier:

- [user.service.ts#L142-L187](#)

The same `user.urId` column is also written by the trusted UR onboarding flow from the provider response:

- [ur-onboarding.service.ts#L108-L137](#)

This creates two authorities for the UR account binding: the external UR provider and the authenticated user profile API. An authenticated user can prefill, replace, or clear their local `urId` independently of the provider-backed `UrAccount` record, leaving local user state inconsistent with the actual UR onboarding state.

This should be narrowed to `urId`. Although `rainId` is also accepted by `updateProfile`, I do not see a separate provider-backed writer for `rainId` in this codebase, so the same dual-authority claim is not established for `Rain`.

Recommendation

Remove `urId` from client profile updates and reserve it for UR onboarding code that writes provider-backed state. If users need to see the identifier, expose it as read-only profile data.

Keep `User.urId` and `UrAccount.urId` updates atomic so local account state cannot diverge from the provider-created UR account.

Techdollar: Resolved with [@95f3f88 ...](#)

Zenith: Verified.

[L-26] Error-state consistency failure in usePaginatedApiQuery

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [useApiQuery.ts](#)
- [applications.action.ts](#)
- [page.tsx](#)
- [RecentActivity.tsx](#)

Description

Paginated request failure must not be rendered as a successful empty dataset. I traced `getApplications` and `getRecentActivity` into `usePaginatedApiQuery`; server actions return unsuccessful response objects, the hook derives `isError`, and both consumers discard that flag while defaulting absent data to `[]` (other authentication queries may still redirect). The `applications` action returns an empty data array on failure, making a transport error resemble a valid empty page. The `activity` action returns an error response without a dataset. Error information is deterministically lost in these views.

```
export default function ApplicationStatusPage() {
  const { data: applications = [], isLoading }
    = usePaginatedApiQuery(EquityQuery.GetApplications())

  return (
    <Transition.Content className="flex h-full flex-col gap-3 md:gap-5">
      <div className="flex flex-col justify-between gap-2 md:flex-row
md:items-center md:gap-2.5">
        <div className="flex flex-col gap-1">
```

An account with 8 stored applications can consequently see 0 < 8 records and the `No Applications Yet` state during a backend, network, or authorization failure. This is misleading. It does not establish persistent authentication bypass, but it can conceal operational faults and misstate account state.

Recommendation

Consume `isError` in every paginated view and render a dedicated error and retry state before empty-state logic. Prefer rejecting failed server actions so React Query preserves transport and authorization failures as errors.

Techdollar: Resolved with [PR-49](#).

Zenith: Verified.

[L-27] Transaction-display consistency failure in toBigNumberish

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [formatter.ts](#)
- [Currency.tsx](#)

Description

An amount rendered by Currency must equal the raw quantity used by transaction builders. I traced `handleChange` into `toBigNumberish`; `parseUnits` computes `bigVal` from the complete input before presentation is shortened, then the component renders `display` while `send`, `approval`, and `swap` paths retain `raw` (wallet confirmation still provides a final check). Transaction builders do not reconstruct the quantity from the string presented to the user. No later interface check restores equality. The two representations can diverge after a long paste. For an 18-decimal asset, pasting `1.0000000000256` leaves a transaction quantity of `1.0000000000256 > 1.00000000002` shown in the field. Supported balances and token supplies bound the hidden suffix, so the proven impact is transaction-intent ambiguity rather than the enormous transfer claimed by the original example.

```
value = integerPart + '.' + decimalPart.slice(0, maxLength)
}

if (value.length > maxLength) {
  value = value.slice(0, maxLength)
}

return { raw: bigVal, formatted: value, display: value.slice(0,
  MAX_DISPLAY_LENGTH) }
```

Recommendation

Normalize and truncate the input before calling `parseUnits`, then derive `raw`, `formatted`, and `display` from that same canonical string. Reject excess precision instead of silently retaining it.

Techdollar: Resolved with [PR-48](#).

Zenith: Verified.

[L-28] `handleSessionExpired` query-isolation invariant failure

SEVERITY: Low**IMPACT:** Medium**STATUS:** Resolved**LIKELIHOOD:** Low.

Target

- [query-client.ts](#)
- [providers.tsx](#)

Description

Authenticated query state must be discarded before a browser tab adopts another principal. I traced `onSessionExpired` to `handleSessionExpired`: the callback removes cookies and performs `router.push`, yet the root `QueryClientProvider` remains mounted around the route tree (no cache operation occurs). `getQueryClient` supplies that provider with one browser singleton, and `handleAuthenticationSuccess` invalidates only `KEYS.AUTH.ME`; equity options such as `ApplicationsQuery.GetApplications` omit user identity. Isolation is not enforced.

```
const handleSessionExpired = () => {
  removeAuthCookie()
  removeReferralCodeCookie()
  router.push(ROUTE_GROUPS.login)
}

return (
  <QueryClientProvider client={queryClient}>
```

For a handoff 512 ms after account A's fetch, the configured `staleTime` comparison is `60000 > 512`; account B's observer therefore reads A's stored response without invoking its query function. I reproduced that state with TanStack's `QueryObserver`. The sequence requires expiration, prompt reauthentication in the same tab, and a matching equity view (ordinary logout reloads the document), but it can expose applications, dashboard metrics, activity, or loans from the previous account.

Recommendation

Clear authenticated queries on session expiry and before completing a new login. Scope every user-specific query key to a stable account identifier as a second isolation boundary.

Techdollar: Resolved with [PR-47](#).

Zenith: Verified.

[L-29] Quote-state consistency failure in Swap

SEVERITY: Low**IMPACT:** Medium**STATUS:** Resolved**LIKELIHOOD:** Low.

Target

- [Swap.tsx](#)

Description

The amount shown when a swap is confirmed must identify the same quote that `handleSwap` submits. I traced `Input.Currency` through `useDebounce` into `GetQuerySwapRoute`; the query key and route parameters lag `amount0` by 300 milliseconds, while `validateSwap` and the action button read current state, leaving the prior `swapRoute` callable during that interval (token changes do clear it). React Query retains the old result while the delayed key remains unchanged. This is a narrow state-transition race. If the existing route encodes 1024 units and the user replaces the field with 512, then $1024 > 512$ can still reach `callFn(swapRoute)` before the debounced query changes. Balance, allowance, simulation, and wallet confirmation remain constraints, but the interface can request a transaction for more than the amount currently presented.

```
SwapQuery.GetQuerySwapRoute(user?.walletAddress || '', {
  from: from.address,
  tokenIn: from.baseToken,
  tokenOut: to.baseToken,
  amount0: debouncedAmount0,
  amount1: debouncedAmount1,
  slippage: DEFAULT_SLIPPAGE,
}),
```

Recommendation

Bind each route to a snapshot of the currently displayed input and disable execution whenever that snapshot differs from `amount0`. Clear the active route immediately on amount edits and re-enable only after a matching quote resolves.

Techdollar: Resolved with [PR-46](#).

Zenith: Verified.

[L-30] Payoff principal-argument inconsistency in PayPrincipal

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [PayPrincipal.tsx](#)
- [blockchain.query.ts](#)
- [eusdte.abi.ts](#)

Description

An eligible on-chain payoff must call `repayOnchainLoanPrincipal()` with its remaining principal while the contract derives the final debt transfer itself. `PayPrincipal` instead combines `loanPrincipal` with a one-percent fee and selected interest, stores that sum in `amount`, and submits it unchanged through `RepayOnchainLoanInterestPrincipal()` (despite the routine's name). I traced `PayPrincipal` -> `RepayOnchainLoanInterestPrincipal()` -> `repayOnchainLoanPrincipal()`; no frontend query obtains the contract's current principal before that state transition. The guard precedes every token transfer. With a remaining principal of 1024 and no selected interest, the fixed fee makes the default amount $1034.24 > 1024$, so the exact-principal guard raises `PartialPrincipalRepaymentNotAllowed` before `safeTransferFrom`. Repayment state and borrower funds remain unchanged. Borrowers using the default modal path cannot close a mature, otherwise eligible on-chain loan; a manually entered exact principal may succeed when every contract check and the larger balance and allowance requirements are met.

```
    return { fee: baseFee + interest, unpaidInterest: interest }
  }, [loan])
const totalAmount = loanPrincipal + fee
const defaultAmount = toBigNumberish(totalAmount, T0.decimals)
const [amount, setAmount] = useState<BigNumberish>(defaultAmount)
const [processing, setProcessing] = useState(false)
```

Recommendation

Query `getOnchainLoanDebt` before submission, pass only its `principal` to `repayOnchainLoanPrincipal`, and render the displayed `totalToClose` as non-editable. Use `totalToClose`

for USDte balance and allowance checks.

Techdollar: Resolved with [PR-44](#).

Zenith: Verified.

[L-31] Authorization inconsistency between Unstake and requestWithdrawal()

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [Unstake.tsx](#)
- [validator.ts](#)
- [blockchain.query.ts](#)

Description

An unstake transition must require only the permissions consumed by requestWithdrawal(). Unstake queries allowance(user, eUSDte, USDTE), and both validateUnstake() and handleAction() treat a shortfall as an instruction to call Approve() before RequestWithdrawal() can run. I traced handleAction() to RequestWithdrawal() through Unstake.tsx and blockchain.query.ts (the approval branch ends the first click); the contract call passes only amount as its function argument under the connected user's account and never uses that eUSDte-to-USDTE permission.

```
export function validateUnstake(params: ValidateUnstakeParams) {
  const { amount0, amount1, balances, allowance, from, isConnected,
    isSupported } = params
  const prefix = 'Unstake'
  const isValidInput = validateInput(amount0, amount1)
  const isApproved = !!allowance && allowance.raw ≥ amount0.raw
  const isValidAmount = !!balances && amount0.raw ≤
    balances[from.address].raw
  const isValid = isValidInput && isApproved && isValidAmount
```

For a caller entering 1024 eUSDte against zero existing approval, $1024 > 0$ makes isApproved false and causes an approval transaction despite the later self-withdrawal requiring none. Any caller lacking enough prior approval pays an avoidable transaction fee and leaves an unneeded eUSDte authorization to USDTE; no direct principal loss is established, yielding Low impact but Medium likelihood for the standard unstake path.

Recommendation

Remove allowance-dependent validation and dispatch from `Unstake`, then submit `RequestWithdrawal()` once the amount, balance, wallet, and network checks pass.

Techdollar: Resolved with [PR-43](#).

Zenith: Verified.

[L-32] urActionStatus consistency loss in UrOnboardingService

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [ur.controller.ts](#)
- [ur-onboarding.service.ts](#)
- [sumsub.service.ts](#)

Description

Final UR submission should require the same live eligibility that allowed onboarding to start, but `consent()` authorizes only from saved Form A readiness. I traced `POST /ur/onboarding/consent` into `UrOnboardingService.consent()`; `start()` applies `KycGuard` and checks `urActionStatus`, while the final method reads `UrAccount.onboardingState`, form hash, session, and UR id without consulting `user.kycStatus` or `user.urActionStatus` (a stale state token is enough). Sumsub webhooks can later mark the user rejected.

```
async consent(user: User): Promise<{ success: boolean; state: string }> {
  const urAccount = await this.urAccountRepository.findOne({ where: {
    userId: user.id } });
  if (!urAccount || urAccount.onboardingState !== 'sign_form_a') {
    throw new ForbiddenException('Form A is not ready for signing');
  }
  if (!urAccount.formATextHash || !urAccount.sessionId || !urAccount.urId) {
    throw new ForbiddenException('Missing required onboarding data');
  }
}
```

If eligibility changes after `sign_form_a`, `1 > 0` still holds for one final submission path over zero allowed live states. The check is historical. A rejected or no-longer-verified user can move the local account to submitting, and possibly advance provider-side onboarding, despite failing the live compliance gate used earlier.

Recommendation

Apply KycGuard to the consent route and repeat the kycStatus/urActionStatus checks inside consent () before calling UR. Re-read the current user row or refresh status so stale request objects cannot authorize final submission.

Techdollar: Resolved with [@7317f2e ...](#)

Zenith: Verified.

[L-33] verifySignature accepts replayable raw-body signatures

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [ur-webhook.service.ts](#)
- [ur-webhook.controller.ts](#)
- [ur-onboarding.service.ts](#)

Description

UR webhook processing should accept each signed provider event once, with freshness included in the signed material. `verifySignature()` validates `verifyMessage()` over only `rawBody.toString('utf8')`, and it checks `deadline` only if the header exists; I traced `handleUrWebhook()` into `handleEvent()` and found no nonce, event ID, or durable replay table before state mutation. Removing an expired deadline leaves 2 > 1 accepted inputs when only one fresh delivery should be valid. A stale `fma.kyc.retry_required` or rejection event can be replayed later to overwrite `UrAccount` onboarding fields and force users back into `retry` or `failed` states.

```
if (deadline) {
  const deadlineTs = parseInt(deadline, 10);
  if (Number.isFinite(deadlineTs) && deadlineTs < Math.floor(Date.now()
    / 1000)) {
    this.logger.warn('UR webhook deadline expired');
    return false;
  }
}
```

Recommendation

Require the deadline or timestamp inside the signed message and reject absent, expired, or malformed freshness fields. Persist a provider event identifier or body digest so `handleEvent()` can ignore already-processed deliveries before updating `UrAccount`.

Techdollar: Resolved with [@6eddfb7...](#)

Zenith: Verified.

[L-34] LoansReadService top-level pagination bypassed by nested payments

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [loans-read.service.ts](#)
- [admin-loan.mapper.ts](#)
- [equity-application-query.service.ts](#)
- [loan-onchain-repayment-sync.service.ts](#)

Description

Paginated loan and application APIs should bound serialized child collections, yet several read paths eager-load payments and then return the whole relation. I traced `LoansReadService.getLoans` and `admin` application profiles into their mappers; the parent query applies `take` and `skip`, while `mapBorrowerLoan` and `mapAdminLoan` sort every loaded payment row for each selected loan (nested cardinality escapes pagination).

```
export function mapBorrowerLoan(loan: Loan) {
  const sortedPayments = [...(loan.payments ?? [])].sort((a, b)
    => new Date(a.paidAt ?? a.createdAt ?? 0).getTime()
    - new Date(b.paidAt ?? b.createdAt ?? 0).getTime());
  const sortedSchedule = [...(loan.schedule ?? [])].sort((a, b)
    => new Date(a.scheduledDate ?? 0).getTime()
    - new Date(b.scheduledDate ?? 0).getTime());
  const openSchedule = sortedSchedule.filter((row)
    => OPEN_SCHEDULE_STATUSES.includes(row.status as any));
  const fullRepayQuote = computeFullRepayAmount(loan, openSchedule);

  return {
```

If one active loan accumulates 1024 repayment events and appears in a page of 20 loans, $20 + 1024 = 1044$ serialized objects can be forced before normal fields are counted. The separate paginated `getPayments` endpoint shows the intended ledger boundary, but the list and detail responses bypass it.

Recommendation

Remove full payments relations from loan/application list and detail projections; return aggregates or a small recent slice, and require the existing paginated payment endpoint for ledger history.

Techdollar: Resolved with [@e36a995 ...](#)

Zenith: Verified.

[L-35] Loan confirmation state transition without KycGuard

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium.

Target

- [equity.controller.ts](#)
- [equity-applications.service.ts](#)
- [kyc.guard.ts](#)

Description

Borrower loan acceptance should require live KYC, but the approved to confirmed transition is reachable through an update route that lacks KycGuard. I traced `equity.controller.ts` into `equity-applications.service.ts`; `createApplication` is guarded, while `updateApplication` accepts status: `confirmed` and calls `confirmApprovedApplication()` without checking refreshed Sumsb state (the user's compliance status can change after approval). The service then creates the loan and registers it on-chain.

```
@Roles('user')
@Put('applications/:id')
@HttpCode(HttpStatus.OK)
@ApiBearerAuth('Turnkey-auth')
@ApiOperation({ summary: 'Update equity application (only draft status)' })
@ApiBody({ type: UpdateEquityApplicationDto })
@ApiResponse({ status: 200, description: 'Application updated successfully', type: EquityApplicationUpdateResponseDto })
@ApiResponse({ status: 400, description: 'Cannot update application', type: ErrorResponseDto })
async updateApplication(
```

A borrower can reach $1 > 0$, with one confirmed loan acceptance and zero current KYC approvals, if their status was revoked after the earlier approval. The bypass is narrow. It still crosses the lending trust boundary, because `loanCreation` and `registerLoanVerification()` execute for a user whose current `kycStatus` may be pending or rejected.

Recommendation

Apply KycGuard to the borrower confirmation route or perform an equivalent service-level refresh immediately before `confirmApprovedApplication` mutates state. Reject the transition unless the current user status is verified at that moment.

Techdollar: Resolved with [@9ee811c ...](#)

Zenith: Verified.

[L-36] Loan activation is committed before repayment schedule creation

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [loan.service.ts](#)

Description:

The `finalizeDisbursement` saves the loan as active before generating its repayment schedule. If schedule creation fails, the initiation worker catches the error, but later runs `select only pending_disbursement_execution` loans and direct retries return early for active loans.

The resulting loan has no payment due date and is also excluded from overdue and default processing.

```
if ((activeLoanStatuses as readonly string[]).includes(loan.status)) {
  if (loan.disbursementTxHash && loan.disbursementTxHash !==
    disbursementTxHash)
  {
    this.logger.warn(
      `Loan ${loanId} already disbursed (status=${loan.status}) with
      disbursementTxHash ${loan.disbursementTxHash}; ignoring new hash
      ${disbursementTxHash}`,
    );
  }
  return loan;
}

if (loan.status !== 'pending_disbursement_execution') {
  throw new Error(`Cannot finalize disbursement for loan in status:
    ${loan.status}`);
}

loan.status = 'active';
loan.disbursementTxHash = disbursementTxHash;
loan.disbursedAt = input.disbursedAt;
if (input.disbursementWalletAddress) {
  loan.disbursementWalletAddress = input.disbursementWalletAddress;
}
```

```
}  
await this.loanRepository.save(loan);  
  
await this.generatePaymentSchedule(loan.id);
```

Recommendations:

Generate the schedule and activate the loan in one database transaction. Add recovery logic that detects active loans without schedule rows and recreates the schedule before treating them as successfully finalized.

Techdollar: Resolved with [@ddfabee ...](#)

Zenith: Verified.

4.4 Informational

A total of 6 informational findings were identified.

[I-1] Turnkey linking consistency loss through updateProfile

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [update-profile.dto.ts](#)
- [user.service.ts](#)

Description

An email used for account linking should represent verified mailbox control, but `User.email` can be written by any authenticated profile owner before Turnkey linkage trusts it. I traced `updateProfile()` into `getOrCreateUser()`; the first path stores an arbitrary unclaimed address after uniqueness checks, while the second path rebinds an existing row when Turnkey later returns the same email (the state variable has two trust levels). That collapses profile metadata and identity proof into one column.

```
if (email) {
  const existingUser = await this.userRepository.findOne({
    where: { email },
  });
  if (existingUser) {
    existingUser.turnkeyUserId = turnkeyUserId;
    existingUser.turnkeyOrganizationId = turnkeyOrganizationId;
    await this.userRepository.save(existingUser);
  }
}
```

One attacker-controlled profile plus one victim Turnkey login gives $1 + 1 = 2$ identities bound to the attacker's seeded row. The flaw is not just duplication. The victim can inherit attacker-selected `walletAddress`, `urId`, `rainId`, and referral state, so later custody and payout decisions can follow data that never came from the verified user.

Recommendation

Separate unverified contact email from verified login email, and require a mailbox-verification ceremony before any value can participate in Turnkey account linking. Avoid overwriting `turnkeyUserId` from an email match unless the link was explicitly initiated and confirmed by the current account owner.

Techdollar: Resolved with [@141b8b3 ...](#)

Zenith: Verified.

[I-2] linkedin scheme validation invariant failure

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [create-waitlist.dto.ts](#)
- [update-waitlist.dto.ts](#)
- [waitlist.service.ts](#)

Description

Profile-link storage must admit only web navigation schemes before values reach operator-facing waitlist data. I traced the unauthenticated create DTO and the admin update DTO; both use `Joi.string().uri()` for `linkedin`, and the service persists the normalized string without checking protocol (the field name does not restrict the parser). The admin read path then returns the stored entity. If 1 public signup stores 1 non-web URI, then 1 + 1 = 2 trust boundaries crossed before an admin interface renders the link. Any frontend that treats `linkedin` as a clickable URL can hand an operator session to attacker-controlled navigation or script execution.

```
export const createWaitlistSchema = Joi.object({
  firstName: Joi.string().trim().min(1).max(255).required(),
  lastName: Joi.string().trim().min(1).max(255).required(),
  email: Joi.string().trim().lowercase().email().max(255).required(),
  company: Joi.string().trim().min(1).max(255).required(),
  job: Joi.string().trim().min(1).max(255).required(),
  linkedin: Joi.string().trim().uri().max(255).optional().allow(null,
    ''),
});
```

Recommendation

Constrain `linkedin` to `https` and `http` in both DTOs, and reject or clear all other schemes before persistence. Treat stored waitlist profile links as untrusted when rendering admin views.

Techdollar: Resolved with [@da14e6c ...](#)

Zenith: Verified.

[I-3] maxLtvPercent policy invariant violation

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [create-company.dto.ts](#)
- [update-company.dto.ts](#)
- [company-valuation.service.ts](#)

Description

Company-level LTV must remain inside platform lending policy, but the create and update DTOs accept any integer and downstream valuation treats the stored number as authoritative. I traced `createCompanySchema` and `updateCompanySchema` into `resolveMaxLtvPercent()` and `computeMaxLoanAmount()`; neither side clamps to `platform.maxLtvPercent`, so a management write becomes borrower-visible loan capacity (the config ceiling is only decorative here).

```
computeMaxLoanAmount(collateralValue: number, maxLtvPercent: number):  
  number {  
    return parseFloat(((collateralValue * maxLtvPercent) / 100).toFixed(2));  
  }  
  
resolveMaxLtvPercent(company: { maxLtvPercent?: number | null },  
  override?: number | null, configDefault?: number): number {  
  return override ?? company.maxLtvPercent ?? configDefault ?? 50;  
}
```

The percentage is policy. If collateral is 512 and stored LTV is 256, $1310.72 > 512$ can be exposed as maximum principal, creating offers and loans above collateral value instead of limiting them to the configured cap.

Recommendation

Validate `maxLtvPercent` on company creation and update against a bounded range and the configured platform maximum. Also clamp or reject in valuation and loan creation so persisted policy drift cannot bypass the input schema.

Techdollar: Resolved with [@28d2a13 ...](#)

Zenith: Verified.

[I-4] SubgraphClientService timeout invariant loss

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [subgraph-client.service.ts](#)
- [run-exclusive.ts](#)

Description

A synchronization guard must release for every cron attempt, but SubgraphClientService supplies an Apollo transport with no deadline while runExclusive waits for completion before clearing isRunning. I traced syncLoanInitiations() -> fetchInitiationEvents() -> client.query() through loan-initiation-sync.service.ts; the same shared client is used by repayment and confirmation polling, so a non-returning GraphQL request holds the per-job flag (one process, one stuck flag). The lock persists. With a stalled endpoint, 1 > 0 pending cron ticks can be skipped while the original await fn() never reaches finally, leaving loan initiation, on-chain repayment, and Safe confirmation state behind the chain.

```
if (!apiKey) {
  throw new Error('Subgraph API key is not configured
    (SUBGRAPH_API_KEY).');
}

this.client = new ApolloClient({
  link: new HttpLink({ uri, headers: { 'api-key': apiKey } }),
  cache: new InMemoryCache(),
  defaultOptions: {
```

Recommendation

Give the shared subgraph transport a hard timeout or abort signal and make the sync jobs treat timeout as a recoverable failure. Keep runExclusive bounded by that deadline so liveness does not depend on a remote endpoint closing the socket.

Techdollar: Resolved with [@7a4a0d6 ...](#)

Zenith: Verified.

[I-5] APP_GUARD ordering weakens authentication throttling

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [app.module.ts](#)
- [turnkey-auth.guard.ts](#)

Description

Authentication attempts should consume rate-limit capacity before expensive credential verification can reject them. AppModule registers ProxyAwareThrottlerGuard only after FrontendApiKeyGuard, TurnkeyAuthGuard, and RolesGuard; I traced GET /user/me through TurnkeyAuthGuard.canActivate() and Nest stops guard evaluation when the bearer token fails (before the counter changes). A client with the shared frontend key can send 1024 malformed session JWTs while the throttle records 0 rejections, so $1024 > 0$ authentication failures bypass the intended boundary accounting. That leaves repeated verifySessionJwtSignature() and decodeSessionJwt() work available as a brute-force and resource-consumption lane.

```
providers: [  
  {  
    provide: APP_GUARD,  
    useClass: FrontendApiKeyGuard,  
  },  
  {  
    provide: APP_GUARD,  
    useClass: TurnkeyAuthGuard,  
  },  
]
```

Recommendation

Register the throttling guard before authentication guards, or apply a pre-authentication limiter to protected routes that rejects before Turnkey verification. Keep the auth bucket aligned with failed login/session checks, not only requests that already authenticated.

Techdollar: Resolved with [@1e60727 ...](#)

Zenith: Verified.

[I-6] Route-schema validation absence in querySwapRoute

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [swap.action.ts](#)
- [swap.query.ts](#)

Description

A swap quote must satisfy the route schema before any transaction path consumes it. I traced querySwapRoute into SwapQuery.GenericSwap: every HTTP body is parsed as JSON and spread into SwapResponse without checking response.ok, then BigInt(data.outputAmount) and data.encodedSwapData are used downstream (contract simulation still prevents the evidence from proving fund loss). Because the Server Action resolves, React Query caches the object as ordinary quote data and no boundary verifies route completeness before transaction construction. Provider errors therefore masquerade as quotes.

```
try {
  const response = await fetch(`${url}?${_params.toString()}`, {
    method: 'GET',
    headers: { 'X-Client-Id': 'TechDollar', Accept: '*/*' },
  }).then((res) => res.json())

  return { ...response, inputAmount: amount.toString(), slippage:
    params.slippage }
```

For a response missing outputAmount and encodedSwapData, 2 > 0 required fields are absent while querySwapRoute still resolves successfully. The next conversion or simulation throws, so a rate-limit response or provider schema change disables quoting instead of producing a controlled error state.

Recommendation

Reject non-success HTTP statuses and validate the decoded payload against an explicit runtime schema before constructing SwapResponse. Convert provider failures into a typed quote error for the query layer.

Techdollar: Resolved with [PR-50](#).

Zenith: Verified.